

C PROGRAMMING / CHAPTERS 1-4 HANDBOOK

從完整題目開始， 把解題思路落實成可測試的程式。

保留第 1-4 章原稿的所有範例，依題目、解題說明、完整程式及測試資料分區閱讀；版型與操作方式統一為第 5-7 章手冊。

53 完整原稿範例	53 可獨立編譯的 C 程式	133 完整輸入 / 輸出測試資料	122 原稿標示的對照投影片頁數
---------------------------------	--------------------------------------	---	--

● 本次 **Linux** 原稿資料重跑 GCC 133/133 Clang 133/133 GCC + UBSan 133/133 判準與觀察值限制 →

內容保留：原有 53 題的題目、解法、程式與 133 組資料均保留；E01-E53 對應原稿閱讀順序，不重新編排檔名。五個主題群組只供介面篩選，不另推定原始章節邊界。

適用範圍：原稿的平台前提、刻意警告及「原稿與新增內容的界線」逐題保留。

跳至主要內容

分辨三類錯誤並執行修正版

16_diagnostics_safe.c

1 組原稿測試

原稿索引：16_diagnostics_safe.c 原檔：4 新版：題目 6 / 講解 7

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

用漏分號、整數除法與格式型別不符三例，建立「先分類、再修正」流程。

輸入與限制

固定 n=3；不需輸入；不可執行的反例只放註解。

應產生的輸出

n=3、兩種比例的差異、正確浮點輸出。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

n=3
integer scale=0.000000
floating scale=0.555556
correct floating value=3.0

78 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容

末尾加分號，讓程式能完成編譯。

STEP 02

保留合法但不合題意的 $5/9$ ，與 $5.0/9.0$ 比較。

STEP 03

`printf` 的格式要配資料型別；錯誤 `%f/int` 反例不執行。

原稿解題示例

$5/9 \rightarrow 0.000000$; $5.0/9.0 \rightarrow 0.555556$

03 完整程式碼

每份程式均有獨立 main

16_diagnostics_safe.c

完整程式 · 20 行 · 內容與原稿逐字相同

```
/* 16_diagnostics_safe.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 4.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    /* Syntax counterexample: int n = 3    (missing semicolon) */
    int n = 3;
    double integer_scale = 5 / 9;
    double floating_scale = 5.0 / 9.0;
    printf("n=%d\n", n);
    printf("integer scale=%.6f\n", integer_scale);
    printf("floating scale=%.6f\n", floating_scale);
    /* Undefined counterexample: printf("%f", 3); */
    printf("correct floating value=%.1f\n", 3.0);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/16_diagnostics_safe.c -o example
```

[跳至主要內容](#)

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E01-T01 1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

n=3
integer scale=0.000000
floating scale=0.555556
correct floating value=3.0

78 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

原稿與新增內容的界線

原第4頁是分類反例；本次新增完整安全對照程式。漏分號與不匹配格式僅留註解，不冒稱原樣可執行。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

建立並執行第一支 C 程式

01_hello.c

1 組原稿測試

原稿索引：01_hello.c 原檔：5, 6 新版：題目 8 / 講解 9, 10

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

將一個 .c 原始檔建置成執行檔，執行後輸出一行問候文字。

輸入與限制

不需輸入；使用 hosted C 環境。

應產生的輸出

Hello, C!，接著換行。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
(空輸入檔，直接 EOF)	Hello, C!
0 bytes · 空輸入檔，直接 EOF	10 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先準備入口：加入 stdio.h，定義 int main(void)。

跳至主要內容

將輸出放在 main：printf 印字串，\n 結束該行。

STEP 03

分開檢查建置與執行：成功編譯後才執行產生的檔案。

原稿解題示例

執行 → Hello, C!

03 完整程式碼

每份程式均有獨立 main

01_hello.c

完整程式 · 12 行 · 內容與原稿逐字相同

```
/* 01_hello.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 5, 6.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    printf("Hello, C!\n");
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/01_hello.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E02-T01 1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

跳至主要內容

核對

STDIN · 完整輸入

STDOUT · 原稿完整預期輸出

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

Hello, C!

10 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原第5頁用 hello.c 示意工具鏈；本包統一實際檔名為 01_hello.c。保留原第6頁程式本體。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

以名稱與註解表達計算意圖

17_naming.c

1 組原稿測試

原稿索引：17_naming.c 原檔：7 新版：題目 11 / 講解 12

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

用合法識別字、巨集及註解完成31分鐘的單位數計算。

輸入與限制

固定 n=31；新增 total_count 初值0。

應產生的輸出

MAX_COUNT=30、blocks=2、total_count=2。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

MAX_COUNT=30 blocks=2 total_count=2
value=3.5

46 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

選能代表資料的名稱，避免把減號或空白塞入識別字。

跳至主要內容

用巨集命名30；以註解說明未滿一單位仍需進位。

STEP 03

保留完整 token，輸出中間結果以核對名稱與資料。

原稿解題示例

31 → (31+29)/30 → 2 blocks

03 完整程式碼

每份程式均有獨立 main

17_naming.c

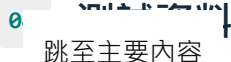
完整程式 · 21 行 · 內容與原稿逐字相同

```
/* 17_naming.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 7.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#define MAX_COUNT 30

int main(void)
{
    int total_count = 0;
    int n = 31;
    float value = 3.5f; /* The token float must not be split. */
    // Round up partial blocks.
    int blocks = (n + MAX_COUNT - 1) / MAX_COUNT;
    total_count += blocks;
    printf("MAX_COUNT=%d blocks=%d total_count=%d\n", MAX_COUNT, blocks, total_count);
    printf("value=%.1f\n", value);
    /* Invalid tokens are not executed: float value; */
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/17_naming.c -o example
```



跳至主要內容

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	MAX_COUNT=30 blocks=2 total_count=2 value=3.5
0 bytes · 空輸入檔 · 直接 EOF	46 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原頁是分散片段；本次用固定n=31與value=3.5f接成微型實驗，這些固定值是新增。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

觀察型別與實作範圍

18_basic_types.c

1 組原稿測試

原稿索引：18_basic_types.c 原檔：8 新版：題目 13 / 講解 14

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

建立int、float、double三種資料，並查看本機int上下界。

輸入與限制

固定3、26.75f、5.0/9.0；範圍由limits.h提供。

應產生的輸出

三種值與本機INT_MIN、INT_MAX。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 原稿平台觀察值

count=3 temp=26.75 ratio=0.555556
INT_MIN=-2147483648 INT_MAX=2147483647

73 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

此例含平台觀察值，請依原題前提判讀，不將本機型別範圍或浮點字面表示當成跨平台固定答案。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

跳至主要內容

依資料性質選型別：數量、溫度、比例各自宣告。

STEP 02

用正確printf規格輸出，float作引數時提升為double。

STEP 03

從limits.h取得整數界線，不把本機結果當作所有平台規格。

原稿解題示例

```
count=3 ; temp=26.75 ; ratio≈0.555556
```

03 完整程式碼

每份程式均有獨立 main

18_basic_types.c

完整程式 · 17 行 · 內容與原稿逐字相同

```
/* 18_basic_types.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 8.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int count = 3;
    float temp = 26.75f;
    double ratio = 5.0 / 9.0;
    printf("count=%d temp=%.2f ratio=%.6f\n", count, temp, ratio);
    printf("INT_MIN=%d INT_MAX=%d\n", INT_MIN, INT_MAX);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/18_basic_types.c -o example
```

0 跳至主要內容 ↓

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

E04-T011. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 平台觀察紀錄

返回碼 0 | 依模式或數值核對；以下列出本次GCC環境觀察值

STDIN · 完整輸入	STDOUT · 原稿平台觀察值
(空輸入檔 · 直接 EOF)	count=3 temp=26.75 ratio=0.555556 INT_MIN=-2147483648 INT_MAX=2147483647
0 bytes · 空輸入檔 · 直接 EOF	73 bytes · 保留原稿的空白、tab 與換行

上方是原 HTML 記錄的 GCC 觀察值，本次環境輸出與其一致；不是所有平台都必須逐字相同的答案。原稿未附具體模式 / 數值判準，本次未補寫該判準。

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

補入原備忘錄提及的limits.h實測，不固定假設int為32位元。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

安全觀察初始化與賦值

19_initialization.c

1 組原稿測試

原稿索引：19_initialization.c 原檔：9 新版：題目 15 / 講解 16

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

依原例 int height, length, width=10，追蹤每個變數何時才可讀。

輸入與限制

初始只有width=10；依序指定height=8、length=12。

應產生的輸出

先只印width，再印已被賦值的變數。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

after declaration: width=10
after height assignment: height=8 width=10
final: height=8 length=12 width=10

106 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

看每個宣告子：`=10`只屬於width，不會初始化所有變數。

跳至主要內容

STEP 02

在height被賦值前不讀它；length亦同。

STEP 03

全部取得有效值後才同時輸出三個變數。

原稿解題示例

width=10 → height=8 → length=12

03 完整程式碼

每份程式均有獨立 main

19_initialization.c

完整程式 · 17 行 · 內容與原稿逐字相同

```
/* 19_initialization.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 9.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int height, length, width = 10;
    printf("after declaration: width=%d\n", width);
    height = 8;
    printf("after height assignment: height=%d width=%d\n", height, width);
    length = 12;
    printf("final: height=%d length=%d width=%d\n", height, length, width);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/19_initialization.c -o example
```

0 跳至主要內容 ↓

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

—

E05-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	after declaration: width=10 after height assignment: height=8 width=10 final: height=8 length=12 width=10
0 bytes · 空輸入檔 · 直接 EOF	106 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原表中的「未指定」不是可印出的數值；本程式刻意不讀尚未初始化的物件。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

用中間值追蹤體積計算

20_volume_flow.c

1 組原稿測試

原稿索引：20_volume_flow.c 原檔：10 新版：題目 17 / 講解 18

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

計算 $12 \times 10 \times 8$ ，再改高為9，觀察已存下的volume是否自動更新。

輸入與限制

固定length=12、width=10、height=8；之後height=9。

應產生的輸出

原體積960、未重算仍960、重算後1080。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

```
initial volume=960
before recompute=960
after recompute=1080
```

61 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容
!值，乘法結果再指定给volume。

STEP 02

改height不等於重跑體積公式；先印舊volume確認。

STEP 03

再次執行賦值，才將1080寫入volume。

原稿解題示例

960 → 改height後仍960 → 重算1080

03 完整程式碼

每份程式均有獨立 main

20_volume_flow.c

完整程式 · 18 行 · 內容與原稿逐字相同

```
/* 20_volume_flow.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 10.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int length = 12, width = 10, height = 8;
    int volume = length * width * height;
    printf("initial volume=%d\n", volume);
    height = 9;
    printf("before recompute=%d\n", volume);
    volume = length * width * height;
    printf("after recompute=%d\n", volume);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/20_volume_flow.c -o example
```

[跳至主要內容](#)

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

—

E06-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

initial volume=960
before recompute=960
after recompute=1080

61 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

後兩次觀察實作原備忘錄的height=9提問，不改變體積公式或單位。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

計算包裹的材積重量

02_dweight.c

7 組原稿測試

原稿索引：02_dweight.c 原檔：11, 13 新版：題目 19 / 講解 20, 23

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

讀入長、寬、高，依教材係數將體積換算為向上取整的材積重量。

輸入與限制

三個整數，依序為長、寬、高；各 1..30 英吋。

應產生的輸出

Volume (in^3) 與 Dimensional weight (lb) 。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

12 10 8

8 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Volume: 960 in^3
Dimensional weight: 6 lb

42 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

確認資料前提：三個整數都成功讀入，且各在 1..30。

跳至主要內容

先算體積：volume = length * width * height，保留可追蹤中間值。

STEP 03

最後進位：以 $(\text{volume} + 165) / 166$ 計算整數磅數。

原稿解題示例

12 10 8 → 960 in³ → 6 lb

03 完整程式碼

每份程式均有獨立 main

02_dweight.c

完整程式 · 27 行 · 內容與原稿逐字相同

```
/* 02_dweight.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 11, 12, 13.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#define INCHES_PER_POUND 166

int main(void)
{
    int length, width, height;
    int volume, weight;
    if (scanf("%d%d%d", &length, &width, &height) != 3) {
        puts("Input error");
        return 1;
    }
    if (length < 1 || length > 30 || width < 1 || width > 30 ||
        height < 1 || height > 30) {
        puts("Range error");
        return 1;
    }
    volume = length * width * height;
    weight = (volume + INCHES_PER_POUND - 1) / INCHES_PER_POUND;
    printf("Volume: %d in^3\n", volume);
    printf("Dimensional weight: %d lb\n", weight);
    return 0;
}
```

單題編譯指令 (使用說明)

```
{ 跳至主要內容 -Wextra -Wpedantic code/02_dweight.c -o example
```

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E07-T01

1. 測試1

exit 0 ✓

原稿測試 · 8 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

12 10 8

8 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Volume: 960 in^3
Dimensional weight: 6 lb

42 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E07-T02

2. 測試2

exit 0 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

1 1 1

6 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Volume: 1 in^3
Dimensional weight: 1 lb

40 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E07-T03

3. 測試3

exit 0 ✓

原稿測試 · 9 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

30 30 30

跳至主要內容

9 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Volume: 27000 in^3
Dimensional weight: 163 lb

46 bytes · 保留原稿的空白、tab 與換行

—

E07-T04

4. 測試4

exit 1 ✓

原稿測試 · 7 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

0 10 8

7 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E07-T05

5. 測試5

exit 1 ✓

原稿測試 · 8 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

31 10 8

8 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E07-T06

6. 測試6

exit 1 ✓

原稿測試 · 7 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

a 10 8

7 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Input error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

跳至主要內容 ↱

exit 1 ✓

式 · 0 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入
(空輸入檔 · 直接 EOF)
0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出
Input error
12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

新增 main、標頭與原投影片已聲明的回傳值及範圍檢查；運算式、單位與係數不變。166 為教材係數，非現行物流費率。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

驗證整數向上取整的邊界

21_ceiling_division.c

8 組原稿測試

原稿索引：21_ceiling_division.c 原檔：12 新版：題目 21 / 講解 22

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

以固定除數166比較直接除法與向上取整公式。

輸入與限制

非負整數 n ，0..30000；166為固定正數。

應產生的輸出

$n/166$ 與 $(n+165)/166$ 。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 範例輸出

floor=0 ceil=0

15 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先限制 n ，使加165仍在保守int範圍內。

跳至主要內容

直接除法只留下整數商；加165將部分單位推到下一格。

STEP 03

測0、165、166、167、332，看整除點有無多算。

原稿解題示例

167 → floor=1 ; ceil=2

03 完整程式碼

每份程式均有獨立 main

21_ceiling_division.c

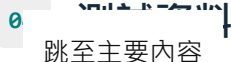
完整程式 · 21 行 · 內容與原稿逐字相同

```
/* 21_ceiling_division.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 12.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int n;
    if (scanf("%d", &n) != 1) {
        puts("Input error");
        return 1;
    }
    if (n < 0 || n > 30000) {
        puts("Range error");
        return 1;
    }
    printf("floor=%d ceil=%d\n", n / 166, (n + 165) / 166);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/21_ceiling_division.c -o example
```



跳至主要內容

8 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E08-T01 1. 測試1

exit 0 ✓

原稿測試 · 2 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

floor=0 ceil=0

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：PASS

E08-T02 2. 測試2

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

165

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

floor=0 ceil=1

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：PASS

E08-T03 3. 測試3

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

166

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

floor=1 ceil=1

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：PASS

E08-T04 4. 測試4

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

跳至主要內容

返回碼 0 | 完整字串核對

STDIN · 完整輸入
167
4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出
floor=1 ceil=2
15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E08-T05

5. 測試5

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入
332
4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出
floor=2 ceil=2
15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E08-T06

6. 測試6

exit 0 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入
30000
6 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出
floor=180 ceil=181
19 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E08-T07

7. 測試7

exit 1 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入
跳至主要內容
3 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出
Range error
12 bytes · 保留原稿的空白、tab 與換行

E08-T08

8. 測試8

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
abc	Input error
4 bytes · 最後有 LF 換行	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

把原表改成可自行輸入n的實驗；本次明示新增 $n \leq 30000$ 界線，避免加165溢位。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

比較巨集有無括號的結果

22_macro_parentheses.c

1 組原稿測試

原稿索引：22_macro_parentheses.c 原檔：14 新版：題目 24 / 講解 25

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

對照3乘BASE的兩種替換方式，並驗證具名材積常數。

輸入與限制

BASE_BAD=10+2、BASE_GOOD=(10+2)、
volume=960。

應產生的輸出

未加括號32、加括號36、weight=6。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

bad=32 good=36 weight=6

24 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先作文字替換：3*BASE_BAD變成3*10+2。

跳至主要內容

STEP 02

再看括號保護：3*(10+2)才是先加後乘。

STEP 03

用INCHES_PER_POUND命名166，讓進位公式可讀。

原稿解題示例

```
3*BASE_BAD=32 ; 3*BASE_GOOD=36
```

03 完整程式碼

每份程式均有獨立 main

22_macro_parentheses.c

完整程式 · 19 行 · 內容與原稿逐字相同

```
/* 22_macro_parentheses.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 14.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#define BASE_BAD 10 + 2
#define BASE_GOOD (10 + 2)
#define INCHES_PER_POUND 166

int main(void)
{
    int bad = 3 * BASE_BAD;
    int good = 3 * BASE_GOOD;
    int volume = 960;
    int weight = (volume + INCHES_PER_POUND - 1) / INCHES_PER_POUND;
    printf("bad=%d good=%d weight=%d\n", bad, good, weight);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/22_macro_parentheses.c -o example
```

[跳至主要內容](#)

04

測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E09-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 完整字串核對	
STDIN · 完整輸入 (空輸入檔 · 直接 EOF) 0 bytes · 空輸入檔 · 直接 EOF	STDOUT · 原稿完整預期輸出 bad=32 good=36 weight=6 24 bytes · 保留原稿的空白、tab 與換行
原稿測試標記： PASS	

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

為能在同檔比較，將原先兩段BASE分別命名BASE_BAD/BASE_GOOD；替換內容與數值不變。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

華氏溫度轉攝氏溫度

03_celsius.c

9 組原稿測試

原稿索引：03_celsius.c 原檔：15, 16 新版：題目 26 / 講解 27, 28

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

依 $C = (F - 32) \times (5/9)$ 計算攝氏；除法發生前必須使用浮點型別。

輸入與限制

一個可表示的 double；本次新增教學範圍
-1000..10000，且須為有限值。

應產生的輸出

Celsius: 攝氏值顯示一位小數。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

212

4 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Celsius: 100.0

15 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

以 %lf 讀入 double；先排除解析失敗、非有限值與超出範圍。

跳至主要內容

STEP 02

先減 32.0，再乘 (5.0 / 9.0)，避免整數 5 / 9 變成 0。

STEP 03

用 %.1f 顯示；以 32、98.6、212 三個已知答案核對。

原稿解題示例

212 → Celsius: 100.0

03 完整程式碼

每份程式均有獨立 main

03_celsius.c

完整程式 · 25 行 · 內容與原稿逐字相同

```
/* 03_celsius.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 15, 16.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#include <math.h>
#define FREEZING_PT 32.0
#define SCALE_FACTOR (5.0 / 9.0)

int main(void)
{
    double fahrenheit, celsius;
    if (scanf("%lf", &fahrenheit) != 1) {
        puts("Input error");
        return 1;
    }
    if (!isfinite(fahrenheit) || fahrenheit < -1000.0 || fahrenheit > 10000.0) {
        puts("Range error");
        return 1;
    }
    celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;
    printf("Celsius: %.1f\n", celsius);
    return 0;
}
```

[跳至主要內容](#)

[單題編譯指令 \(使用說明 \)](#)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/03_celsius.c -o example
```

04

測試資料

9 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

–

E10-T01

1. 測試1

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

212

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Celsius: 100.0

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E10-T02

2. 測試2

exit 0 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

32

3 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Celsius: 0.0

13 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E10-T03

3. 測試3

exit 0 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

跳至主要內容

98.6

STDOUT · 原稿完整預期輸出

Celsius: 37.0

5 bytes · 最後有 LF 換行

14 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E10-T04

4. 測試4

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

-40

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Celsius: -40.0

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E10-T05

5. 測試5

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

abc

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Input error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E10-T06

6. 測試6

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

nan

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

跳至主要內容

—

E10-T07

7. 測試7

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

[跳至主要內容](#)

驗證基本元素的四題檢核

23_quiz_basics.c

1 組原稿測試

原稿索引：23_quiz_basics.c 原檔：17 新版：題目 29 / 講解 30

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

用可執行結果確認初始化、除法、巨集；建置流程以文字作答。

輸入與限制

固定c=3；a、b未初始化時不讀取。

應產生的輸出

c、整數比例與巨集結果；列出#include的階段。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

c=3 x=0.0 macro=32
after assignments: a=0 b=0
#include is processed during preprocessing.

90 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先判定哪些變數可讀，只印有初值的c。

跳至主要內容

STEP 02

保留5/9與未括號巨集，觀察合法但容易誤判的值。

STEP 03

#include的處理屬建置過程，不假裝它是main內的執行動作。

原稿解題示例

```
c=3 ; x=0.0 ; 3*BASE=32
```

03 完整程式碼

每份程式均有獨立 main

23_quiz_basics.c

完整程式 · 19 行 · 內容與原稿逐字相同

```
/* 23_quiz_basics.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 17.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#define BASE 10 + 2

int main(void)
{
    int a, b, c = 3;
    double x = 5 / 9;
    printf("c=%d x=%.1f macro=%d\n", c, x, 3 * BASE);
    /* Assign before observing a and b. */
    a = 0; b = 0;
    printf("after assignments: a=%d b=%d\n", a, b);
    puts("#include is processed during preprocessing.");
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/23_quiz_basics.c -o example
```

[跳至主要內容](#)

04

測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E11-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字符串核對

返回碼 0 完整字符串核對	
STDIN · 完整輸入 (空輸入檔 · 直接 EOF) 0 bytes · 空輸入檔 · 直接 EOF	STDOUT · 原稿完整預期輸出 c=3 x=0.0 macro=32 after assignments: a=0 b=0 #include is processed during preprocessing. 90 bytes · 保留原稿的空白、tab 與換行
原稿測試標記： PASS	

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

新增a=b=0僅為示範使用前赋值，不改變原題「只有c已初始化」的答案。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

讓格式與數值一一配對

24_printf_contract.c

1 組原稿測試

原稿索引：24_printf_contract.c 原檔：18 新版：題目 31 / 講解 32

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

輸出quantity=3與temperature=26.75，依序使用整數及一位小數格式。

輸入與限制

固定int quantity與double temperature。

應產生的輸出

Qty=3 Temp=26.8。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

Qty=3 Temp=26.8

16 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先看%d要取int值，讓quantity放第一個位置。

跳至主要內容

再看%.1f要取double值，讓temperature放第二個位置。

STEP 03

一般文字與空白照印；最後由\n換行。

原稿解題示例

quantity=3 、 temperature=26.75 → Qty=3 Temp=26.8

03 完整程式碼

每份程式均有獨立 main

24_printf_contract.c

完整程式 · 14 行 · 內容與原稿逐字相同

```
/* 24_printf_contract.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 18.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int quantity = 3;
    double temperature = 26.75;
    printf("Qty=%d Temp=%.1f\n", quantity, temperature);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/24_printf_contract.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

–

跳至主要內容

式 · 0 input bytes · 完整字串核對

exit 0 ✓

返回碼 0 | 完整字串核對

STDIN · 完整輸入
(空輸入檔 · 直接 EOF)
0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出
Qty=3 Temp=26.8
16 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

補齊原圖所列的變數宣告及main。顯示預期沿用一般最近捨入環境。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

依資料型別選擇 printf 規格

25_printf_types.c

1 組原稿測試

原稿索引：25_printf_types.c 原檔：19 新版：題目 33 / 講解 34

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

將原表中的int、unsigned、字元、浮點與字串各印一次。

輸入與限制

固定42、42u、A、3.5f、3.5、Hello。

應產生的輸出

使用各自匹配規格的六行輸出。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

int=42
unsigned=42
char=A
float=3.5
double=3.5
string=Hello

60 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

跳至主要內容

按C型別而非數值外觀選%d、%u或%f。

STEP 02

字元用%c、字串用%s；兩者不是同一種參數。

STEP 03

float傳入printf後提升為double，不要把讀入指標規則混入。

原稿解題示例

3.5f與3.5 → 都以printf的%f輸出

03 完整程式碼

每份程式均有獨立 main

25_printf_types.c

完整程式 · 22 行 · 內容與原稿逐字相同

```
/* 25_printf_types.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 19.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int n = 42;
    unsigned int u = 42u;
    float f = 3.5f;
    double d = 3.5;
    printf("int=%d\n", n);
    printf("unsigned=%u\n", u);
    printf("char=%c\n", 'A');
    printf("float=%.1f\n", f);
    printf("double=%.1f\n", d);
    printf("string=%s\n", "Hello");
    /* Counterexample (not executed): printf("%d", 3.0); */
    return 0;
}
```

單題編譯指令 (使用說明)

跳至主要內容

```
{ -Wextra -Wpedantic code/25_printf_types.c -o example
```

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E13-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

int=42
unsigned=42
char=A
float=3.5
double=3.5
string=Hello

60 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

原稿與新增內容的界線

表格具體化，型別不匹配的原反例只保留在註解。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

比較欄寬、精度與浮點格式

04_printf_formats.c

1 組原稿測試

原稿索引：04_printf_formats.c 原檔：20, 21 新版：題目 35 / 講解 36, 37

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

對同一整數及同一浮點值，觀察不同格式如何改變顯示，而不改變儲存值。

輸入與限制

固定 i=40、x=839.21；不需輸入。

應產生的輸出

以 | 標記欄位邊界，並列出 %f、%e、%g。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
(空輸入檔，直接 EOF)	
0 bytes · 空輸入檔，直接 EOF	40 40 040 00040 123456 839.210 8.392e+02 839.2 839.210
	78 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依 I 跳至主要內容 R 只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先算整數位數：欄寬補空白，整數精度補前導零。

STEP 02

以同一浮點值比較固定小數、科學記號與有效位數。

STEP 03

測超出欄寬的 123456：最小欄寬不會截掉數字。

原稿解題示例

%5.3d、40 → | 040|

03 完整程式碼

每份程式均有獨立 main

04_printf_formats.c

完整程式 · 22 行 · 內容與原稿逐字相同

```
/* 04_printf_formats.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 20, 21.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 40;
    double x = 839.21;
    printf("|%5d|\n", i);
    printf("|%-5d|\n", i);
    printf("|%5.3d|\n", i);
    printf("|%05d|\n", i);
    printf("|%5d|\n", 123456);
    printf("%.3f\n", x);
    printf("%.3e\n", x);
    printf("%.4g\n", x);
    printf("|%10.3f|\n", x);
}
```

跳至主要內容

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/04_printf_formats.c -o example
```

04

測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E14-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 完整字串核對	
STDIN · 完整輸入 (空輸入檔 · 直接 EOF) 0 bytes · 空輸入檔 · 直接 EOF	STDOUT · 原稿完整預期輸出 40 40 040 0040 123456 839.210 8.392e+02 839.2 839.210 78 bytes · 保留原稿的空白、tab 與換行
原稿測試標記： PASS	

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

將原兩頁表格包成可執行實驗；新增 123456 作最小欄寬的反證。浮點字面測試採本次一般二進位浮點、C locale 環境。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

製作感測器數值報表

05_sensor_report.c

1 組原稿測試

原稿索引：05_sensor_report.c 原檔：22 新版：題目 38 / 講解 39

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

將兩筆假設量測值輸出成名稱、固定小數與科學記號三欄。

輸入與限制

固定 Temp=26.75、Volt=3.30；無外接感測器。

應產生的輸出

三欄標題與兩筆對齊資料。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

Item	Value	Scientific
Temp	26.75	2.675e+01
Volt	3.30	3.300e+00

93 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

為每欄決定規格：名稱靠左，數值欄靠右。

跳至主要內容

STEP 02

同一筆數值分別交給 `%8.2f` 與 `%12.3e` 顯示。

STEP 03

以標題與兩筆資料檢查對齊，不用手打空白湊位置。

原稿解題示例

Temp → 26.75 → 2.675e+01

03 完整程式碼

每份程式均有獨立 main

05_sensor_report.c

完整程式 · 16 行 · 內容與原稿逐字相同

```
/* 05_sensor_report.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 22.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    double temperature = 26.75;
    double voltage = 3.30;
    printf("%-8s %8s %12s\n", "Item", "Value", "Scientific");
    printf("%-8s %8.2f %12.3e\n", "Temp", temperature, temperature);
    printf("%-8s %8.2f %12.3e\n", "Volt", voltage, voltage);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/05_sensor_report.c -o example
```

0 跳至主要內容 ↓

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

Item	Value	Scientific
Temp	26.75	2.675e+01
Volt	3.30	3.300e+00

93 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

沿用原第22頁完整程式，未新增外部資料來源。對齊以 ASCII 與等寬終端顯示為前提。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

控制字串截取與動態欄寬

26_dynamic_format.c

1 組原稿測試

原稿索引：26_dynamic_format.c 原檔：23 新版：題目 40 / 講解 41

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

分別顯示SENSOR前四個ASCII字元，以及由參數控制寬度與精度的3.5。

輸入與限制

字串SENSOR；width=8、precision=2、value=3.5。

應產生的輸出

|SENS |、| 3.50|、單一百分號。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
(空輸入檔，直接 EOF)	SENS 3.50 %
0 bytes · 空輸入檔，直接 EOF	24 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

%-8.4s先限制最多四個位元組，再靠左補滿八格。

跳至主要內容

STEP 02

%*.f依序取width、precision、value，共三個參數。

STEP 03

%%直接產生百分號，不再拿取一個數值。

原稿解題示例

8, 2, 3.5 → | 3.50|

03 完整程式碼

每份程式均有獨立 main

26_dynamic_format.c

完整程式 · 14 行 · 內容與原稿逐字相同

```
/* 26_dynamic_format.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 23.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    printf("|%-8.4s|\n", "SENSOR");
    printf("|%*.f|\n", 8, 2, 3.5);
    printf("%%\n");
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/26_dynamic_format.c -o example
```

04 測試資料

1 組完整原稿資料

輸，跳至主要內容 ↵ 用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	SENS 3.50 %
0 bytes · 空輸入檔 · 直接 EOF	24 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

保留原兩段printf；新增百分號示範。此ASCII資料不推廣為Unicode可見字數計算。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

輸出引號、反斜線與換行

27_escape_sequences.c

1 組原稿測試

原稿索引：27_escape_sequences.c 原檔：24 新版：題目 42 / 講解 43

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

用跳脫序列印出原案例的引號文字與Windows風格路徑。

輸入與限制

不需輸入。

應產生的輸出

第一行為"C"\tmp，第二行為C:\temp。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

"C"\tmp
C:\temp

16 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先分清字串界定引號與真正要輸出的雙引號。

跳至主要內容

用\\表示單一反斜線，避免\t被當成跳格。

STEP 03

用\n終止該行；來源寫法與顯示結果分兩層核對。

原稿解題示例

來源 \" 與 \\ → 顯示 " 與 \

03 完整程式碼

每份程式均有獨立 main

27_escape_sequences.c
完整程式 · 13 行 · 內容與原稿逐字相同

```
/* 27_escape_sequences.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 24.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    printf("\"C\"\\tmp\n");
    printf("C:\\temp\n");
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/27_escape_sequences.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E17-T01 1. 測試1

exit 0 ✓

式 · 0 input bytes · 完整字串核對

跳至主要內容

返回碼 0 | 完整字串核對

STDIN · 完整輸入
(空輸入檔 · 直接 EOF)
0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出
"C"\\tmp C:\\temp
16 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

第二行來自原備忘錄的路徑提問。沒有用system或作業系統專屬函式。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

區分多給參數與危險格式錯誤

28_printf_extra_args.c

1 組原稿測試

原稿索引：28_printf_extra_args.c 原檔：25 新版：題目 44 / 講解 45

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

只執行原表中可定義的多餘參數例，再用不同變數觀察其求值。

輸入與限制

固定i=10、j=20；不執行少參數或型別不匹配反例。

應產生的輸出

只印格式需要的i；多餘+j仍把j改成21。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
(空輸入檔，直接 EOF)	first=10 second=10 j=21
0 bytes · 空輸入檔，直接 EOF	24 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先排除不能給固定輸出的兩種反例，只留註解。

跳至主要內容

STEP 02

printf只消耗格式需要的參數；其餘參數仍會求值。

STEP 03

將j的結果在下一行印出，避免同次呼叫的衝突讀寫。

原稿解題示例

printf只印10；額外++j仍使j=21

03 完整程式碼

每份程式均有獨立 main

28_printf_extra_args.c

完整程式 · 18 行 · 內容與原稿逐字相同

```
/* 28_printf_extra_args.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 25.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 10, j = 20;
    /* The next two calls intentionally trigger format-extra-args warnings. */
    printf("first=%d\n", i, j);
    printf("second=%d\n", i, ++j);
    printf("j=%d\n", j);
    /* DO NOT execute: printf("%d %d", i); */
    /* DO NOT execute: printf("%f", i); */
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/28_printf_extra_args.c -o example
```

跳至主要內容 示「刻意保留診斷對照」。程式中的教學反例與警告沒有因格式轉換而刪除；詳細界線見本題第 05 區。

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E18-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字符串核對

返回碼 0 | 完整字符串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

first=10
second=10
j=21

24 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

原稿與新增內容的界線

原頁多餘參數例保留；新增++j讓「仍求值」可觀察。此檔刻意觸發format-extra-args警告，預期警告不是編譯失敗。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

正確讀取 float 與 double

29_scan_value_address.c

4 組原稿測試

原稿索引：29_scan_value_address.c 原檔：26 新版：題目 46 / 講解 47

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

各讀一個浮點數，配對目標物件型別，再以printf輸出。

輸入與限制

依序一個float與一個double；示例3.5 3.5。

應產生的輸出

f=3.500000 d=3.500000。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
3.5 3.5	f=3.500000 d=3.500000
8 bytes · 最後有 LF 換行	22 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

兩個目標分別宣告float f與double d。

跳至主要內容

scanf用%f寫入&f、用%lf寫入&d；成功兩項才繼續。

STEP 03

輸出取的是值，兩者皆以printf的%f表示。

原稿解題示例

3.5 3.5 → f=3.500000 d=3.500000

03 完整程式碼

每份程式均有獨立 main

29_scan_value_address.c

完整程式 · 18 行 · 內容與原稿逐字相同

```
/* 29_scan_value_address.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 26.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    float f;
    double d;
    if (scanf("%f%lf", &f, &d) != 2) {
        puts("Input error");
        return 1;
    }
    printf("f=%f d=%f\n", f, d);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/29_scan_value_address.c -o example
```

04 測試資料

4 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列I 跳至主要內容 結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

–

E19-T01

1. 測試1

exit 0 ✓

原稿測試 · 8 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
3.5 3.5	f=3.500000 d=3.500000
8 bytes · 最後有 LF 換行	22 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E19-T02

2. 測試2

exit 0 ✓

原稿測試 · 7 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
-2.5 8	f=-2.500000 d=8.000000
7 bytes · 最後有 LF 換行	23 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E19-T03

3. 測試3

exit 1 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
3.5 x	Input error
6 bytes · 最後有 LF 換行	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E19-T04

4. 測試4

exit 1 ✓

原稿測試 · 0 input bytes · 完整字串核對

跳至主要內容

返回碼 1 | 完整字串核對

STDIN · 完整輸入
(空輸入檔 · 直接 EOF)
0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出
Input error
12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

將原頁兩次scanf合併成一次兩項讀取，格式與物件配對不變；不將不同大小的指標互換。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

逐項解析沒有空白的數字串

06_scan_stream.c

5 組原稿測試

原稿索引：06_scan_stream.c 原檔：27, 28 新版：題目 48 / 講解 49, 50

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

使用 %d%d%f 讀四個數字，再觀察下一個未消耗的字元。

輸入與限制

例如 1-20.3-4.0e3，或以空白 / 換行分隔的同一組值。

應產生的輸出

成功項數、四個變數與下一個字元。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

1-20.3-4.0e3

13 bytes · 最後有 LF 換行

STDOUT · 範例輸出

count=4
i=1 j=-20 x=0.3 y=-4000.0
next=NEWLINE

47 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

依格式次序切割：第一個 %d 讀 1，第二個讀 -20。

跳至主要內容

STEP 02

兩個 %f 接著讀 .3 與 -4.0e3；四項都成功才印數值。

STEP 03

用 getchar 額外觀察剩餘字元；換行不是被 scanf 自動吃掉。

原稿解題示例

1-20.3-4.0e3 → 1, -20, 0.3, -4000.0

03 完整程式碼

每份程式均有獨立 main

06_scan_stream.c

完整程式 · 25 行 · 內容與原稿逐字相同

```
/* 06_scan_stream.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 27, 28.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 0, j = 0;
    float x = 0.0f, y = 0.0f;
    int count = scanf("%d%d%f%f", &i, &j, &x, &y);
    if (count == EOF) puts("count=EOF");
    else printf("count=%d\n", count);
    if (count != 4) {
        puts("Input error");
        return 1;
    }
    printf("i=%d j=%d x=%.1f y=%.1f\n", i, j, x, y);
    int next = getchar();
    if (next == EOF) puts("next=EOF");
    else if (next == '\n') puts("next=NEWLINE");
    else printf("next=%c\n", next);
    return 0;
}
```

[跳至主要內容](#)

[單題編譯指令 \(使用說明 \)](#)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/06_scan_stream.c -o example
```

04

測試資料

5 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E20-T01

1. 測試1

exit 0 ✓

原稿測試 · 13 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

1-20.3-4.0e3

13 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

count=4
i=1 j=-20 x=0.3 y=-4000.0
next=NEWLINE

47 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E20-T02

2. 測試2

exit 0 ✓

原稿測試 · 18 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

1
-20 .3
-4.0e3

18 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

count=4
i=1 j=-20 x=0.3 y=-4000.0
next=NEWLINE

47 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E20-T03

3. 測試3

exit 0 ✓

原稿測試 · 12 input bytes · 完整字串核對

跳至主要內容 

STDIN · 完整輸入

STDOUT · 原稿完整預期輸出

1-20.3-4.0e3

12 bytes · 最後沒有換行 · 直接 EOF

count=4
i=1 j=-20 x=0.3 y=-4000.0
next=EOF

43 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E20-T04

4. 測試4

exit 1 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

12 x

5 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

count=1
Input error

20 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E20-T05

5. 測試5

exit 1 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

count=EOF
Input error

22 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05 來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

新增宣告、初值、回傳值檢查及 getchar 觀察器。getchar 僅供看見剩餘輸入，不是清空輸入的操作。

本[跳至主要內容]措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

觀察 scanf 部分成功後的狀態

07_scan_failure.c

4 組原稿測試

原稿索引：07_scan_failure.c 原檔：29 新版：題目 51 / 講解 52

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

讀取三個整數；故意讓第二項失敗，確認哪些變數已改、哪些仍保留初值。

輸入與限制

標準示例 12 x 3；另測完全成功、首項錯誤、EOF。

應產生的輸出

count、a/b/c 與 next；本程式是診斷實驗。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
12 x 3	count=1
	a=12 b=-1 c=-1
	next=x
7 bytes · 最後有 LF 換行	30 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先把三個變數都設為 -1，才能安全觀察未賦值項目。

跳至主要內容

STEP 02

執行一次 `scanf`，保留成功賦值數；不要失敗後繼續算總和。

STEP 03

用 `getchar` 看下一個字元；`x` 不會被下一次 `%d` 自動略過。

原稿解題示例

12 x 3 → count=1 · a=12 · b=c=-1 · next=x

03 完整程式碼

每份程式均有獨立 main

07_scan_failure.c

完整程式 · 21 行 · 內容與原稿逐字相同

```
/* 07_scan_failure.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 29.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = -1, b = -1, c = -1;
    int count = scanf("%d%d%d", &a, &b, &c);
    if (count == EOF) puts("count=EOF");
    else printf("count=%d\n", count);
    printf("a=%d b=%d c=%d\n", a, b, c);
    int next = getchar();
    if (next == EOF) puts("next=EOF");
    else if (next == '\n') puts("next=NEWLINE");
    else if (next == ' ') puts("next=SPACE");
    else printf("next=%c\n", next);
    return 0;
}
```

單題編譯指令（使用說明）

{ 跳至主要內容 -Wextra -Wpedantic code/07_scan_failure.c -o example

04

測試資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E21-T01 1. 測試1 原稿測試 · 7 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 12 x 3 7 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 count=1 a=12 b=-1 c=-1 next=x 30 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E21-T02 2. 測試2 原稿測試 · 6 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 1 2 3 6 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 count=3 a=1 b=2 c=3 next=NEWLINE 33 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E21-T03 3. 測試3 原稿測試 · 2 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 跳至主要內容 2 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 count=0 a=-1 b=-1 c=-1 next=x	

30 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

— E21-T04 4. 測試4

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

count=EOF
a=-1 b=-1 c=-1
next=EOF

34 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05 來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

這是觀察 scanf 失敗的測試器，預期失敗仍回傳程序結束碼0，表示觀察流程正常完成；不將 count 與程序結束碼混為一談。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

重現五組分隔符匹配實驗

30_scan_delimiters.c

1 組原稿測試

原稿索引：30_scan_delimiters.c 原檔：30 新版：題目 53 / 講解 54

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

用固定字串重現原表五組scanf格式的成功項數，避免互相消耗同一輸入。

輸入與限制

固定5/6、5 / 6與5 6；本次用sscanf作獨立測試器。

應產生的輸出

五次成功賦值數依序2、1、2、2、1。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 範例輸出

case1: count=2 a=5 b=6
case2: count=1 a=5 b=-1
case3: count=2 a=5 b=6
case4: count=2 a=5 b=6
case5: count=1 a=5 b=-1

117 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容 記；格式空白可略過零個或多個空白。

STEP 02

每次重設a、b初值，失敗後仍可安全查看未改值。

STEP 03

用sscanf從不同字串開頭開始，保留相同轉換與匹配規則。

原稿解題示例

%d/%d讀5 / 6 → count=1，b仍-1

03 完整程式碼

每份程式均有獨立 main

30_scan_delimiters.c

完整程式 · 26 行 · 內容與原稿逐字相同

```
/* 30_scan_delimiters.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 30.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = -1, b = -1, count;
    count = sscanf("5/6", "%d/%d", &a, &b);
    printf("case1: count=%d a=%d b=%d\n", count, a, b);
    a = -1; b = -1;
    count = sscanf("5 / 6", "%d/%d", &a, &b);
    printf("case2: count=%d a=%d b=%d\n", count, a, b);
    a = -1; b = -1;
    count = sscanf("5 / 6", "%d / %d", &a, &b);
    printf("case3: count=%d a=%d b=%d\n", count, a, b);
    a = -1; b = -1;
    count = sscanf("5/6", "%d / %d", &a, &b);
    printf("case4: count=%d a=%d b=%d\n", count, a, b);
    a = -1; b = -1;
    count = sscanf("5 6", "%d, %d", &a, &b);
    printf("case5: count=%d a=%d b=%d\n", count, a, b);
    return 0;
}
```

[跳至主要內容](#)

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/30_scan_delimiters.c -o example
```

04

測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E22-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	case1: count=2 a=5 b=6 case2: count=1 a=5 b=-1 case3: count=2 a=5 b=6 case4: count=2 a=5 b=6 case5: count=1 a=5 b=-1
0 bytes · 空輸入檔 · 直接 EOF	117 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

sscanf是本次新增的測試支援，並非原頁已使用。只把資料來源從stdin換成固定字串，不用它模擬終端等待。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

比較 %c 與前導空白的讀取

08_scan_char.c

4 組原稿測試

原稿索引：08_scan_char.c 原檔：31 新版：題目 55 / 講解 56

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

先讀一個整數，再讀一個原始字元與一個跳過空白的字元。

輸入與限制

輸入 123、換行、A、換行。

應產生的輸出

整數、raw 是否為換行、最後讀到的字元。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
123 A	n=123 raw=NEWLINE skipped=A
6 bytes · 最後有 LF 換行	28 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

數值讀取只消耗數字；輸入中的換行仍留在串流。

跳至主要內容

STEP 02

用 %c 讀取 raw，直接觀察那個換行。

STEP 03

再用「空格 + %c」讀下一個非空白字元 A。

原稿解題示例

123↵A ↪ raw=NEWLINE ; skipped=A

03 完整程式碼

每份程式均有獨立 main

08_scan_char.c

完整程式 · 26 行 · 內容與原稿逐字相同

```
/* 08_scan_char.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 31.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int n;
    char raw, skipped;
    if (scanf("%d", &n) != 1) {
        puts("Input error");
        return 1;
    }
    if (scanf("%c", &raw) != 1 || scanf(" %c", &skipped) != 1) {
        puts("Input error");
        return 1;
    }
    printf("n=%d\n", n);
    if (raw == '\n') puts("raw=NEWLINE");
    else if (raw == ' ') puts("raw=SPACE");
    else printf("raw=%c\n", raw);
    printf("skipped=%c\n", skipped);
    return 0;
}
```

[跳至主要內容](#)

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/08_scan_char.c -o example
```

04

測試資料

4 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E23-T01

1. 測試1

exit 0 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

123
A

6 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

n=123
raw=NEWLINE
skipped=A

28 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E23-T02

2. 測試2

exit 0 ✓

原稿測試 · 8 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

123 A

8 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

n=123
raw=SPACE
skipped=A

26 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E23-T03

3. 測試3

exit 1 ✓

原稿測試 · 2 input bytes · 完整字串核對

跳至主要內容 

STDIN · 完整輸入

STDOUT · 原稿完整預期輸出

x

2 bytes · 最後有 LF 換行

Input error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E23-T04

4. 測試4

exit 1 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

123

3 bytes · 最後沒有換行，直接 EOF

STDOUT · 原稿完整預期輸出

Input error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05 來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原頁為兩個替代方案；本次將兩種讀取接在同一示範輸入上，讓它們依序消耗不同字元，並明確印出 raw 與 skipped。不是把同一串流倒回重讀。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

[跳至主要內容](#)

親自觀察尾端空白指令的等待

31_scan_trailing_space.c

3 組原稿測試

原稿索引：31_scan_trailing_space.c 原檔：32 新版：題目 57 / 講解 58

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

分兩階段讀取：第一個%d立即結束；第二個%d\n需繼續看到非空白或EOF。

輸入與限制

先輸入7；再輸入8，最後補X或送出EOF。

應產生的輸出

plain=7；trailing=8；下一個未消耗字元X。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

7
8
X

6 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Stage 1: enter an integer (plain %d).
plain=7
Stage 2: enter another integer, then X or EOF (%d\n).
trailing=8
next=X

118 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容

白的%d，看到第一段完成。

STEP 02

再執行%d\n；只按Enter可能尚未滿足尾端空白指令。

STEP 03

輸入X讓讀取結束，getchar確認X未被空白指令消耗。

原稿解題示例

7↵8↵X↵ → plain=7 · trailing=8 · next=X

03 完整程式碼

每份程式均有獨立 main

31_scan_trailing_space.c

完整程式 · 29 行 · 內容與原稿逐字相同

```
/* 31_scan_trailing_space.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 32.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int plain, trailing;
    puts("Stage 1: enter an integer (plain %d).");
    fflush(stdout);
    if (scanf("%d", &plain) != 1) {
        puts("Input error");
        return 1;
    }
    printf("plain=%d\n", plain);
    puts("Stage 2: enter another integer, then X or EOF (%d\\n).");
    fflush(stdout);
    if (scanf("%d\\n", &trailing) != 1) {
        puts("Input error");
        return 1;
    }
    printf("trailing=%d\\n", trailing);
    int next = getchar();
    if (next == EOF) puts("next=EOF");
    跳至主要內容 printf("next=%c\\n", next);
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/31_scan_trailing_space.c -o example
```

04

測試資料

3 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E24-T01

1. 測試1

exit 0 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

7
8
X

6 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Stage 1: enter an integer (plain %d).
plain=7
Stage 2: enter another integer, then X or EOF (%d\n).
trailing=8
next=X

118 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E24-T02

2. 測試2

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

7
8

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Stage 1: enter an integer (plain %d).
plain=7
Stage 2: enter another integer, then X or EOF (%d\n).
trailing=8
next=EOF

120 bytes · 保留原稿的空白、tab 與換行

[跳至主要內容](#)

E24-T03

3. 測試3

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

abc

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Stage 1: enter an integer (plain %d).
Input error

50 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

這支程式刻意展示互動等待，不是推薦輸入模板。新增fflush(stdout)只確保提示顯示，從未使用fflush(stdin)。自動測試以完整重導輸入或EOF終止；另有互動等待驗證。

互動操作時第二階段只輸入8與Enter可能會等待；補X與Enter即可。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

將兩個分數相加且不先約分

09_fractions.c

8 組原稿測試

原稿索引：09_fractions.c 原檔：33, 34 新版：題目 59 / 講解 60, 61

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

保留整數分子與分母，透過交叉相乘計算和；不先做整數除法。

輸入與限制

依序輸入 $n1/d1$ 與 $n2/d2$ ；分子 $-100..100$ ，分母 $1..100$ 。

應產生的輸出

Sum: numerator/denominator；本章不約分。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

5/6
3/4

8 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Sum: 38/24

11 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先完成兩次 $\%d / \%d$ 讀取；/ 是必須匹配的普通字元。

跳至主要內容

STEP 02

驗證分子、分母範圍，確保乘積與相加不超出保守 int 範圍。

STEP 03

分子交叉相乘後相加，分母相乘；最後輸出兩個整數。

原稿解題示例

$5/6 + 3/4 \rightarrow \text{Sum: } 38/24$

03 完整程式碼

每份程式均有獨立 main

09_fractions.c

完整程式 · 26 行 · 內容與原稿逐字相同

```
/* 09_fractions.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 33, 34.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int n1, d1, n2, d2;
    int numerator, denominator;
    if (scanf("%d / %d", &n1, &d1) != 2 ||
        scanf("%d / %d", &n2, &d2) != 2) {
        puts("Input error");
        return 1;
    }
    if (n1 < -100 || n1 > 100 || n2 < -100 || n2 > 100 ||
        d1 < 1 || d1 > 100 || d2 < 1 || d2 > 100) {
        puts("Range error");
        return 1;
    }
    numerator = n1 * d2 + n2 * d1;
    denominator = d1 * d2;
    printf("Sum: %d/%d\n", numerator, denominator);
    return 0;
}
```

[跳至主要內容](#)

[單題編譯指令 \(使用說明\)](#)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/09_fractions.c -o example
```

04

測試資料

8 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E25-T01

1. 測試1

exit 0 ✓

原稿測試 · 8 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

5/6
3/4

8 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Sum: 38/24

11 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E25-T02

2. 測試2

exit 0 ✓

原稿測試 · 12 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

5 / 6
3 / 4

12 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Sum: 38/24

11 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E25-T03

3. 測試3

exit 0 ✓

原稿測試 · 9 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

跳至主要內容

-1/2

STDOUT · 原稿完整預期輸出

Sum: 0/4

1/2

9 bytes · 最後有 LF 換行

9 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E25-T04 4. 測試4

exit 0 ✓

原稿測試 · 16 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

100/100
100/100

16 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Sum: 20000/10000

17 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E25-T05 5. 測試5

exit 1 ✓

原稿測試 · 8 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

5/0
3/4

8 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E25-T06 6. 測試6

exit 1 ✓

原稿測試 · 10 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

101/2
1/2

跳至主要內容
.F 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

-

E25-T07

7. 測試7

exit 1 ✓

原稿測試 · 8 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
5,6 3/4	Input error
8 bytes · 最後有 LF 換行	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

-

E25-T08

8. 測試8

exit 1 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔，直接 EOF)	Input error
0 bytes · 空輸入檔，直接 EOF	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

保留原投影片的未約分語意、格式與範圍；補上標頭、main、宣告及完整防護。未加入最大公因數或迴圈。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

用小實驗驗證四題格式化檢核

32_quiz_io.c

1 組原稿測試

原稿索引：32_quiz_io.c 原檔：35 新版：題目 62 / 講解 63

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

將欄寬、分隔符失敗、剩餘換行與double輸入分成四個互不干擾的測試。

輸入與限制

皆為固定資料；sscanf是本次新增測試支援。

應產生的輸出

| 040|、count=1、raw為換行、x=3.5。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

| 040|

slash: count=1 a=5 b=-1

char: count=2 n=123 newline=1

double: count=1 x=3.5

84 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容

，計算補零與補空白的不同。

STEP 02

以獨立字串驗證/匹配失敗與數字後的換行。

STEP 03

最後以%lf配double，避免前一組剩餘輸入干擾下一題。

原稿解題示例

四題結果：040前兩格空白、1、換行、%lf

03 完整程式碼

每份程式均有獨立 main

32_quiz_io.c

完整程式 · 22 行 · 內容與原稿逐字相同

```
/* 32_quiz_io.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 35.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    printf("|%5.3d|\n", 40);
    int a = -1, b = -1;
    int count = sscanf("5 / 6", "%d/%d", &a, &b);
    printf("slash: count=%d a=%d b=%d\n", count, a, b);
    int n = 0;
    char ch = '?';
    count = sscanf("123\nA\n", "%d%c", &n, &ch);
    printf("char: count=%d n=%d newline=%d\n", count, n, ch == '\n');
    double x = 0.0;
    count = sscanf("3.5", "%lf", &x);
    printf("double: count=%d x=%.1f\n", count, x);
    return 0;
}
```

單題編譯指令 (使用說明)

跳至主要內容

```
{ Wextra -Wpedantic code/32_quiz_io.c -o example
```

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E26-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

| 040|
slash: count=1 a=5 b=-1
char: count=2 n=123 newline=1
double: count=1 x=3.5

84 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：PASS

原稿與新增內容的界線

將備忘錄答案實作成固定輸入測試；未將本次sscanf測試支援說成原教材程式。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

拆解一元與二元運算

33_unary_binary.c

1 組原稿測試

原稿索引：33_unary_binary.c 原檔：36 新版：題目 64 / 講解 65

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

為原式 $-a+b\times 2$ 指定小整數，分別輸出子式與整體結果。

輸入與限制

本次新增固定 $a=3$ 、 $b=4$ 。

應產生的輸出

$-a=-3$ 、 $b\times 2=8$ 、 $result=5$ 。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

$-a=-3$ $b\times 2=8$ $result=5$

21 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

一元-只取 a ，得到 -3 。

跳至主要內容

二元乘法取 b 與 2 ，得到 8 。

STEP 03

加法組合兩個子結果；不把負號一律當成二元相減。

原稿解題示例

$a=3 \cdot b=4 \rightarrow -3+8=5$

03 完整程式碼

每份程式均有獨立 main

33_unary_binary.c
完整程式 · 14 行 · 內容與原稿逐字相同

```
/* 33_unary_binary.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 36.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = 3, b = 4;
    int result = -a + b * 2;
    printf("-a=%d b*2=%d result=%d\n", -a, b * 2, result);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/33_unary_binary.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— 1 exit 0 ✓

跳至主要內容 式 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

-a=-3 b*2=8 result=5

21 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原頁只列符號a與b；3與4是本次明示加入的測試初值。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

比較兩筆量測值的平均

34_average_division.c

1 組原稿測試

原稿索引：34_average_division.c 原檔：37 新版：題目 66 / 講解 67

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

計算80與81的平均，對照整數除法及在除法前引入浮點數。

輸入與限制

固定a=80、b=81。

應產生的輸出

integer division=80.0；floating division=80.5。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

integer division=80.0
floating division=80.5

45 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先求a+b=161；若除以整數2，商先截為80。

跳至主要內容

改成2.0後，除法才會保留小數而得到80.5。

STEP 03

將兩式分別存入double再輸出，顯示左側型別不會補回小數。

原稿解題示例

(80+81)/2 → 80.0 ; /2.0 → 80.5

03 完整程式碼

每份程式均有獨立 main

34_average_division.c

完整程式 · 16 行 · 內容與原稿逐字相同

```
/* 34_average_division.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 37.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = 80, b = 81;
    double truncated = (a + b) / 2;
    double fractional = (a + b) / 2.0;
    printf("integer division=%.1f\n", truncated);
    printf("floating division=%.1f\n", fractional);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/34_average_division.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

跳至主要內容

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

integer division=80.0
floating division=80.5

45 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

保留原值；原備忘錄16_comma_and_average.c的提及更新為此檔與46_comma_operator.c，拆為兩個獨立範例。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

驗證正負整數的商與餘數

35_quotient_remainder.c

1 組原稿測試

原稿索引：35_quotient_remainder.c 原檔：38 新版：題目 68 / 講解 69

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

列出17與3的四種正負組合，驗證 $a=(a/b)\times b+a\%b$ 。

輸入與限制

固定±17、±3；除數非零且商可表示。

應產生的輸出

每組a、b、q、r及還原值。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

```
17 3: q=5 r=2 rebuild=17
17 -3: q=-5 r=2 rebuild=17
-17 3: q=-5 r=-2 rebuild=-17
-17 -3: q=5 r=-2 rebuild=-17
```

110 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容 商朝0截斷。

STEP 02

餘數配合商還原被除數；非零餘數與被除數同號。

STEP 03

用四次固定計算核對表格，不測除零或不可表示商。

原稿解題示例

$-17/3=-5$ ； $-17\%3=-2$

03 完整程式碼

每份程式均有獨立 main

35_quotient_remainder.c

完整程式 · 19 行 · 內容與原稿逐字相同

```
/* 35_quotient_remainder.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 38.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = 17, b = 3;
    printf("%d %d: q=%d r=%d rebuild=%d\n", a, b, a/b, a%b, (a/b)*b+a%b);
    a = 17; b = -3;
    printf("%d %d: q=%d r=%d rebuild=%d\n", a, b, a/b, a%b, (a/b)*b+a%b);
    a = -17; b = 3;
    printf("%d %d: q=%d r=%d rebuild=%d\n", a, b, a/b, a%b, (a/b)*b+a%b);
    a = -17; b = -3;
    printf("%d %d: q=%d r=%d rebuild=%d\n", a, b, a/b, a%b, (a/b)*b+a%b);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/35_quotient_remainder.c -o example
```

[跳至主要內容](#)

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E29-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	17 3: q=5 r=2 rebuild=17 17 -3: q=-5 r=2 rebuild=17 -17 3: q=-5 r=-2 rebuild=-17 -17 -3: q=5 r=-2 rebuild=-17
0 bytes · 空輸入檔 · 直接 EOF	110 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

原稿與新增內容的界線

以重複敘述而非迴圈呈現四列，避免提前要求後章語法；完全沿用原表數字。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

把總秒數拆成時、分、秒

10_duration.c

9 組原稿測試

原稿索引：10_duration.c 原檔：39 新版：題目 70 / 講解 71

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

將非負持續秒數拆成完整小時、剩餘分鐘與秒數。

輸入與限制

一個整數秒數，0..30000。

應產生的輸出

hours:minutes:seconds；分、秒固定補到兩位。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

3671

5 bytes · 最後有 LF 換行

STDOUT · 範例輸出

1:01:11

8 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先以 /3600 求完整小時，從總量中切出最大單位。

跳至主要內容

用 %3600 保留不足一小時的部分，再 /60 求分鐘。

STEP 03

秒數為 %60 ; 以 %02d 只改顯示，不改值。

原稿解題示例

3671 → 1:01:11

03 完整程式碼

每份程式均有獨立 main

10_duration.c

完整程式 · 24 行 · 內容與原稿逐字相同

```
/* 10_duration.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 39.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int seconds, hours, minutes, remainder;
    if (scanf("%d", &seconds) != 1) {
        puts("Input error");
        return 1;
    }
    if (seconds < 0 || seconds > 30000) {
        puts("Range error");
        return 1;
    }
    hours = seconds / 3600;
    minutes = (seconds % 3600) / 60;
    remainder = seconds % 60;
    printf("%d:%02d:%02d\n", hours, minutes, remainder);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/10_duration.c -o example
```

[跳至主要內容](#)

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E30-T01 1. 測試1 原稿測試 · 5 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 3671 5 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 1:01:11 8 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E30-T02 2. 測試2 原稿測試 · 2 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 0 2 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 0:00:00 8 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E30-T03 3. 測試3 原稿測試 · 3 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 59 跳至主要內容 換行 STDOUT · 原稿完整預期輸出 0:00:59 8 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	

—

E30-T04

4. 測試4

exit 0 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
60	0:01:00
3 bytes · 最後有 LF 換行	8 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E30-T05

5. 測試5

exit 0 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
3600	1:00:00
5 bytes · 最後有 LF 換行	8 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E30-T06

6. 測試6

exit 0 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
30000	8:20:00
6 bytes · 最後有 LF 換行	8 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E30-T07

7. 測試7

exit 1 ✓

原稿測試 · 3 input bytes · 完整字串核對

跳至主要內容

返回碼 1 | 完整字串核對

STDIN · 完整輸入

-1

3 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E30-T08 8. 測試8

exit 1 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

30001

6 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E30-T09 9. 測試9

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

abc

4 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Input error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05 來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

保留原備忘錄給定的0..30000界線；此為持續時間，不加入日期、跨日與時區。

本[跳至主要內容]措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

比較括號改變前後的運算樹

36_precedence.c

1 組原稿測試

原稿索引：36_precedence.c 原檔：40 新版：題目 72 / 講解 73

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

對同一組 a 、 b 、 c ，計算 $a+b*c$ 與 $(a+b)*c$ 。

輸入與限制

固定 $a=2$ 、 $b=3$ 、 $c=4$ 。

應產生的輸出

無額外括號 14；加括號 20。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入	STDOUT · 範例輸出
(空輸入檔，直接 EOF)	$a+b*c=14$ $(a+b)*c=20$
0 bytes · 空輸入檔，直接 EOF	20 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先把 $a+b*c$ 分組為 $a+(b*c)$ ，乘法子樹先形成一個值。

跳至主要內容

括號把另一式改成 $(a+b)*c$ ，得到不同公式。

STEP 03

分別印兩個結果；不由分組推論所有求值時間順序。

原稿解題示例

2+3*4=14 ; (2+3)*4=20

03 完整程式碼

每份程式均有獨立 main

36_precedence.c

完整程式 · 14 行 · 內容與原稿逐字相同

```
/* 36_precedence.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 40.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = 2, b = 3, c = 4;
    printf("a+b*c=%d\n", a + b * c);
    printf("(a+b)*c=%d\n", (a + b) * c);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/36_precedence.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

1

跳至主要內容

式 · 0 input bytes · 完整字串核對

exit 0 ✓

返回碼 0 | 完整字串核對

STDIN · 完整輸入
(空輸入檔 · 直接 EOF)
0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出
a+b*c=14 (a+b)*c=20
20 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

沿用原圖的固定2、3、4與兩種分組。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

用括號驗證結合性

37_associativity.c

1 組原稿測試

原稿索引：37_associativity.c 原檔：41 新版：題目 74 / 講解 75

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

驗證連續減法、乘除以及連鎖賦值的語法分組。

輸入與限制

固定20、5、3、2；i/j/k初始均有值。

應產生的輸出

20-5-3=12；20/5*2=8；連鎖後i=j=k=0。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

20-5-3=12
20-(5-3)=18
20/5*2=8
i=0 j=0 k=0

43 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容

·5)-3，不能補成20-(5-3)。

STEP 02

同級乘除由左分組，先取20/5的商再乘2。

STEP 03

賦值右結合，最內層k=0的值逐層寫到j及i。

原稿解題示例

$$20/5*2 \rightarrow (20/5)*2 \rightarrow 8$$

03 完整程式碼

每份程式均有獨立 main

37_associativity.c

完整程式 · 17 行 · 內容與原稿逐字相同

```
/* 37_associativity.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 41.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 1, j = 2, k = 3;
    printf("20-5-3=%d\n", 20 - 5 - 3);
    printf("20-(5-3)=%d\n", 20 - (5 - 3));
    printf("20/5*2=%d\n", 20 / 5 * 2);
    i = j = k = 0;
    printf("i=%d j=%d k=%d\n", i, j, k);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/37_associativity.c -o example
```

0 跳至主要內容 ↓

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

E32-T011. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	20-5-3=12 20-(5-3)=18 20/5*2=8 i=0 j=0 k=0
0 bytes · 空輸入檔 · 直接 EOF	43 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

新增對照括號結果18與初始化1/2/3，使連鎖更新可觀察；原表結論不變。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

追蹤連鎖賦值的型別轉換

38_assignment_value.c

1 組原稿測試

原稿索引：38_assignment_value.c 原檔：42 新版：題目 76 / 講解 77

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

執行f=i=33.3f，觀察內層賦值先轉int後提供什麼值。

輸入與限制

int i、float f；常數33.3f。

應產生的輸出

i=33、f=33.0。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

i=33 f=33.0

12 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先辨識右結合：f=(i=33.3f)。

跳至主要內容

33.3f寫入int時得到33；內層運算式提供int值33。

STEP 03

外層把33轉成float給f，因此不是原常數33.3。

原稿解題示例

33.3f → int 33 → float 33.0

03 完整程式碼

每份程式均有獨立 main

38_assignment_value.c

完整程式 · 15 行 · 內容與原稿逐字相同

```
/* 38_assignment_value.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 42.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i;
    float f;
    f = i = 33.3f;
    printf("i=%d f=%.1f\n", i, f);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/38_assignment_value.c -o example
```

此範例原稿已標示「刻意保留診斷對照」。程式中的教學反例與警告沒有因格式轉換而刪除；詳細界線見本題第 05 區。

04 測試資料

1 組完整原稿資料

輸出，[跳至主要內容](#) 用原稿；來源觀察值另行標示。

列印版列出的測試系列與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	i=33 f=33.0
0 bytes · 空輸入檔 · 直接 EOF	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原式完整保留。Clang可能就常數33.3f到int的刻意截斷提出警告；此為本題要觀察的轉換。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

辨識可修改的賦值目標

39_lvalue_safe.c

1 組原稿測試

原稿索引：39_lvalue_safe.c 原檔：43 新版：題目 78 / 講解 79

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

只執行合法的i=3；將常數、運算結果及const目標反例作文字分類。

輸入與限制

固定const limit=10；不修改limit。

應產生的輸出

i=3、limit=10，以及反例分類文字。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

```
i=3 limit=10
12=i, i+j=0, -i=j: not modifiable lvalues.
limit=20: the const object cannot be modified.
```

103 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先問左側是否代表一個物件，再問它能否被修改。

跳至主要內容

STEP 02

只對可修改的*i*做賦值，`const`物件保持原值。

STEP 03

不把`12=i`或`i+j=0`拿來執行，因為它們違反語言限制。

原稿解題示例

`i=3`可執行；`12=i`與`limit=20`只作反例

03 完整程式碼

每份程式均有獨立 `main`

39_lvalue_safe.c

完整程式 · 18 行 · 內容與原稿逐字相同

```
/* 39_lvalue_safe.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 43.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 0;
    const int limit = 10;
    i = 3;
    printf("i=%d limit=%d\n", i, limit);
    puts("12=i, i+j=0, -i=j: not modifiable lvalues.");
    puts("limit=20: the const object cannot be modified.");
    /* Invalid counterexamples are intentionally not executable statements. */
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/39_lvalue_safe.c -o example
```

跳至主要內容

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

—

E34-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	<pre>i=3 limit=10 12=i, i+j=0, -i=j: not modifiable lvalues. limit=20: the const object cannot be modified.</pre>
0 bytes · 空輸入檔 · 直接 EOF	103 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原頁以表格列不可編譯反例；本次提供分類器及合法側，不聲稱所有原反例可變成原樣可執行檔。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

追蹤庫存與複合賦值

40_compound_assignment.c

1 組原稿測試

原稿索引：40_compound_assignment.c 原檔：44 新版：題目 80 / 講解 81

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

先處理42件庫存的售出與補貨，再比較*=分組、+=與=+。

輸入與限制

stock=42；售7補5；另設i=2,j=3,k=4。

應產生的輸出

庫存42→35→40；i*=j+k為14；+=得13、=+得3。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

```
start=42
after sale=35
after restock=40
i*=j+k: 14
i+=j: 13
i=+j: 3
```

68 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

跳至主要內容

對同一stock逐步更新，逐行印出新狀態。

STEP 02

$\ast$ =右側的 $j+k$ 視為整組，不可展開成 $i\ast j+k$ 。

STEP 03

比較+=與+=前重設 $i=10$ ，避免把兩個獨立實驗接續計算。

原稿解題示例

$42 - 7 + 5 \rightarrow \text{stock}=40$

03 完整程式碼

每份程式均有獨立 main

40_compound_assignment.c

完整程式 · 24 行 · 內容與原稿逐字相同

```
/* 40_compound_assignment.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 44.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int stock = 42;
    printf("start=%d\n", stock);
    stock -= 7;
    printf("after sale=%d\n", stock);
    stock += 5;
    printf("after restock=%d\n", stock);
    int i = 2, j = 3, k = 4;
    i *= j + k;
    printf("i*=j+k: %d\n", i);
    i = 10; i += j;
    printf("i+=j: %d\n", i);
    i = 10; i += j;
    printf("i+=j: %d\n", i);
    return 0;
}
```

單頁 跳至主要內容 月)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/40_compound_assignment.c -o example
```

04

測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E35-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 完整字串核對	
STDIN · 完整輸入 (空輸入檔 · 直接 EOF) 0 bytes · 空輸入檔 · 直接 EOF	STDOUT · 原稿完整預期輸出 start=42 after sale=35 after restock=40 i*=j+k: 14 i+=j: 13 i+=j: 3 68 bytes · 保留原稿的空白、tab 與換行
原稿測試標記： PASS	

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

庫存與+=例沿用原內容；為*=j+k新增小整數初值2、3、4。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

分開觀察遞增的值與狀態

41_pre_post_increment.c

1 組原稿測試

原稿索引：41_pre_post_increment.c 原檔：45 新版：題目 82 / 講解 83

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

兩次都從i=1開始，分別執行x=++i與x=i++。

輸入與限制

每次實驗重設i=1。

應產生的輸出

前置x=2,i=2；後置x=1,i=2。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

prefix: x=2 i=2

postfix: x=1 i=2

33 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

前置先更新物件，這個運算式提供新值。

跳至主要內容

後置提供原值，但完整敘述結束後i已增加。

STEP 03

在下一個printf才印狀態，不把遞增與讀同一i塞入不同參數。

原稿解題示例

`++i → x=2 ; i++ → x=1 ;` 兩者最後i都2

03 完整程式碼

每份程式均有獨立 main

41_pre_post_increment.c

完整程式 · 17 行 · 內容與原稿逐字相同

```
/* 41_pre_post_increment.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 45.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 1, x;
    x = ++i;
    printf("prefix: x=%d i=%d\n", x, i);
    i = 1;
    x = i++;
    printf("postfix: x=%d i=%d\n", x, i);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/41_pre_post_increment.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列{ 跳至主要內容 結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E36-T01 1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

prefix: x=2 i=2
postfix: x=1 i=2

33 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原兩個獨立例直接補成main，保留每次重設i=1。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

追蹤三組獨立的遞增運算式

11_expression_trace.c

1 組原稿測試

原稿索引：11_expression_trace.c 原檔：46 新版：題目 84 / 講解 85

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

對三組表格範例分別重設 i=1、j=2，再算 k 與最後狀態。

輸入與限制

固定初值；每一組都是獨立實驗。

應產生的輸出

逐組列出 i、j、k。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

case1: i=2 j=3 k=4
case2: i=2 j=3 k=3
case3: i=0 j=1 k=2

57 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先辨識每個 ++ / -- 提供舊值或新值，而不是只看最後狀態。

跳至主要內容

STEP 02

左右更新不同物件，用各自提供的值計算 k。

STEP 03

下一組前重設 i、j，避免沿用上一組已修改的狀態。

原稿解題示例

$++i + j++ \rightarrow i=2 \cdot j=3 \cdot k=4$

03 完整程式碼

每份程式均有獨立 main

11_expression_trace.c

完整程式 · 20 行 · 內容與原稿逐字相同

```
/* 11_expression_trace.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 46.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 1, j = 2, k;
    k = ++i + j++;
    printf("case1: i=%d j=%d k=%d\n", i, j, k);
    i = 1; j = 2;
    k = i++ + j++;
    printf("case2: i=%d j=%d k=%d\n", i, j, k);
    i = 1; j = 2;
    k = --i + j--;
    printf("case3: i=%d j=%d k=%d\n", i, j, k);
    return 0;
}
```

單題編譯指令（使用說明）

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/11_expression_trace.c -o example
```

[跳至主要內容](#)

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E37-T01 1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字符串核對

返回碼 0 | 完整字符串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

case1: i=2 j=3 k=4
case2: i=2 j=3 k=3
case3: i=0 j=1 k=2

57 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

原稿與新增內容的界線

保留原第46頁三組運算；補上重設初值與輸出。此程式不示範同一變數未排序的衝突讀寫。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

逐層解出複合運算式

53_complex_expression.c

1 組原稿測試

原稿索引：53_complex_expression.c 原檔：47 新版：題目 86 / 講解 87

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

依原初值計算 $a=b+=c++-d+--e/-f$ ，再列出六個變數。

輸入與限制

$b=1, c=10, d=10, e=10, f=3$ ； a 只在賦值後讀取。

應產生的輸出

$a=-2, b=-2, c=11, d=10, e=9, f=3$ 。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

$a=-2 \ b=-2 \ c=11 \ d=10 \ e=9 \ f=3$

28 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先標註提供值： $c++$ 給10、 $--e$ 給9、 $-f$ 給-3。

跳至主要內容

算術部分 $10-10+9/(-3)=-3$ ； b 加上它後成-2。

STEP 03

內層賦值提供-2再給a；最後分開印所有狀態。

原稿解題示例

右側-3 → b=-2 → a=-2；c=11、e=9

03 完整程式碼

每份程式均有獨立 main

53_complex_expression.c
完整程式 · 14 行 · 內容與原稿逐字相同

```
/* 53_complex_expression.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 47.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a, b = 1, c = 10, d = 10, e = 10, f = 3;
    a = b += c++ - d + --e / -f;
    printf("a=%d b=%d c=%d d=%d e=%d f=%d\n", a, b, c, d, e, f);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/53_complex_expression.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

1

跳至主要內容

式 · 0 input bytes · 完整字串核對

exit 0 ✓

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

a=-2 b=-2 c=11 d=10 e=9 f=3

28 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原複合式與所有初值均完整保留；不以手算步驟宣稱編譯器必依此順序求值。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

分開驗證分組與人為指定順序

42_grouping_order.c

1 組原稿測試

原稿索引：42_grouping_order.c 原檔：48 新版：題目 88 / 講解 89

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

對(a+b)*(c-d)選定無衝突小整數，計算直接式及兩種明確分步版本。

輸入與限制

本次新增a=2,b=3,c=10,d=4。

應產生的輸出

直接式、先算左式、先算右式皆為30。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 範例輸出

direct=30 left-first=30 right-first=30

39 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

括號固定乘法的兩個運算元；它們只讀不同已初始化資料。

跳至主要內容

用分行敘述明確寫出先左後右與先右後左。

STEP 03

結果相同只因本例沒有相依副作用，不表示編譯器承諾固定次序。

原稿解題示例

$(2+3)*(10-4)=30$ ；兩種分步路徑也都是30

03 完整程式碼

每份程式均有獨立 main

42_grouping_order.c

完整程式 · 20 行 · 內容與原稿逐字相同

```
/* 42_grouping_order.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 48.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = 2, b = 3, c = 10, d = 4;
    int direct = (a + b) * (c - d);
    int left = a + b;
    int right = c - d;
    int left_first = left * right;
    right = c - d;
    left = a + b;
    int right_first = left * right;
    printf("direct=%d left-first=%d right-first=%d\n", direct, left_first, right_first);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/42_grouping_order.c -o example
```

04 測試資料

1 組完整原稿資料

輸, 跳至主要內容 ↵ 用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

direct=30 left-first=30 right-first=30

39 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

本次新增固定初值與兩條明確分行計算。沒有透過執行結果推斷原式編譯器的實際求值順序。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

將四種語言行為分開說明

43_behavior_classes.c

1 組原稿測試

原稿索引：43_behavior_classes.c 原檔：49 新版：題目 90 / 講解 91

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

用實作範圍與無衝突運算實測，其他兩類只保留分類，不執行非法或未定義程式。

輸入與限制

本機limits.h；固定 $(2+3)*(10-4)$ 。

應產生的輸出

int上下界、純運算值30及兩種不可執行的分類。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 原稿平台觀察值

```
INT_MIN=-2147483648 INT_MAX=2147483647
pure expression=30
12=i: constraint violation; not executed.
i*i++: undefined behavior; not executed.
```

141 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

此例含平台觀察值，請依原題前提判讀，不將本機型別範圍或浮點字面表示當成跨平台固定答案。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依 [C 語言標準](#) 只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。
跳至主要內容

STEP 01

限制違反要先修正，不替12=i預測執行輸出。

STEP 02

實作特性查本機界限；未指定順序不等於所有結果都不確定。

STEP 03

i*i++是反例，只記錄分類，不以某次結果當語言保證。

原稿解題示例

INT範圍看本機；純運算30；i*i++不執行

03

完整程式碼

每份程式均有獨立 main

43_behavior_classes.c
完整程式 · 16 行 · 內容與原稿逐字相同

```
/* 43_behavior_classes.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 49.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("INT_MIN=%d INT_MAX=%d\n", INT_MIN, INT_MAX);
    printf("pure expression=%d\n", (2 + 3) * (10 - 4));
    puts("12=i: constraint violation; not executed.");
    puts("i*i++: undefined behavior; not executed.");
    return 0;
}
```

單題編譯指令 (使用說明)

{

跳至主要內容

-Wextra -Wpedantic code/43_behavior_classes.c -o example

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E40-T01 1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 平台觀察紀錄

返回碼 0 | 依模式或數值核對；以下列出本次GCC環境觀察值

STDIN · 完整輸入	STDOUT · 原稿平台觀察值
(空輸入檔 · 直接 EOF)	INT_MIN=-2147483648 INT_MAX=2147483647 pure expression=30 12=i: constraint violation; not executed. i*i++: undefined behavior; not executed.
0 bytes · 空輸入檔 · 直接 EOF	141 bytes · 保留原稿的空白、tab 與換行

上方是原 HTML 記錄的 GCC 觀察值，本次環境輸出與其一致；不是所有平台都必須逐字相同的答案。原稿未附具體模式 / 數值判準，本次未補寫該判準。

原稿測試標記：**PASS**

原稿與新增內容的界線

新增安全分類展示，不試圖將未定義行為列為有限結果集合。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

把衝突副作用改成明確步驟

15_safe_rewrite.c

1 組原稿測試

原稿索引：15_safe_rewrite.c 原檔：50 新版：題目 92 / 講解 93

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01

完整題目

先確認規格，再寫程式

依「先用舊a計算b，再改a」的新明確意圖，重寫原反例。

輸入與限制

固定 a=5；另示範 i=2 時保存舊值後再遞增。

應產生的輸出

a=1、b=7、c=6；第二例 old_i=2、i=3、j=4。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

a=1 b=7 c=6
old_i=2 i=3 j=4

28 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02

解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先說明演算法意圖，而不是替未定義原式猜一個答案。

跳至主要內容

拆成 b=a+2、a=1、c=b-a 三個完整敘述。

STEP 03

需要舊值時先保存，再更新；每一步完成後才讀下一個狀態。

原稿解題示例

$a=5 \rightarrow b=7 \rightarrow a=1 \rightarrow c=6$

03 完整程式碼

每份程式均有獨立 main

15_safe_rewrite.c

完整程式 · 23 行 · 內容與原稿逐字相同

```
/* 15_safe_rewrite.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 50.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int a = 5, b, c;
    /* Original counterexample is intentionally NOT executed:
     * c = (b = a + 2) - (a = 1); */
    b = a + 2;
    a = 1;
    c = b - a;
    printf("a=%d b=%d c=%d\n", a, b, c);
    int i = 2;
    int old_i = i;
    i++;
    int j = old_i * old_i;
    printf("old_i=%d i=%d j=%d\n", old_i, i, j);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/15_safe_rewrite.c -o example
```

跳至主要內容



1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

E41-T011. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	a=1 b=7 c=6 old_i=2 i=3 j=4
0 bytes · 空輸入檔 · 直接 EOF	28 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

第一段沿用原頁右側明確重寫；第二段實作原備忘錄提到的old_i策略。兩者都不是聲稱保有未定義原式的標準結果。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

正確檢查分數是否在範圍內

44_relational_logical.c

6 組原稿測試

原稿索引：44_relational_logical.c 原檔：51 新版：題目 94 / 講解 95

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

比較正確的 $0 \leq \text{score} \ \&\& \ \text{score} \leq 100$ 與原頁的連寫不等式。

輸入與限制

可表示int；教學限制score在-1000..1000。

應產生的輸出

正確range判斷與錯誤連寫判斷，皆用0/1表示。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

-5

3 bytes · 最後有 LF 換行

STDOUT · 範例輸出

score=-5 correct=0 chained=1

29 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

把下界與上界各變成一個比較，再用&&合併。

跳至主要內容

保留原錯誤式觀察：第一個比較先產生0或1。

STEP 03

再和100比較時總成立，故不能代表原數值的範圍。

原稿解題示例

score=-5 → correct=0 · chained=1

03 完整程式碼

每份程式均有獨立 main

44_relational_logical.c

完整程式 · 24 行 · 內容與原稿逐字相同

```
/* 44_relational_logical.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 51.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int score;
    if (scanf("%d", &score) != 1) {
        puts("Input error");
        return 1;
    }
    if (score < -1000 || score > 1000) {
        puts("Range error");
        return 1;
    }
    int correct = 0 <= score && score <= 100;
    /* Intentional logical mistake: observe compiler diagnostics. */
    int chained = 0 <= score <= 100;
    printf("score=%d correct=%d chained=%d\n", score, correct, chained);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/44_relational_logical.c -o example
```

跳至主要內容 示「刻意保留診斷對照」。程式中的教學反例與警告沒有因格式轉換而刪除；詳細界線見本題第 05 區。

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E42-T01 1. 測試1 原稿測試 · 3 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 -5 3 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 score=-5 correct=0 chained=1 29 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E42-T02 2. 測試2 原稿測試 · 2 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 0 2 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 score=0 correct=1 chained=1 28 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E42-T03 3. 測試3 原稿測試 · 4 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 100 跳至主要內容 ：換行 STDOUT · 原稿完整預期輸出 score=100 correct=1 chained=1 30 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	

–

E42-T04

4. 測試4

exit 0 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
101	score=101 correct=0 chained=1
4 bytes · 最後有 LF 換行	30 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E42-T05

5. 測試5

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
abc	Input error
4 bytes · 最後有 LF 換行	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E42-T06

6. 測試6

exit 1 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
1001	Range error
5 bytes · 最後有 LF 換行	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

跳至主要內容

原稿與新增內容的界線

保留原合法但邏輯錯誤的連寫式；編譯器可出現bool-compare等警告，正是教材要辨識的問題。新增-1000..1000輸入界線。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

[跳至主要內容](#)

用短路規則保護除法

45_short_circuit.c

1 組原稿測試

原稿索引：45_short_circuit.c 原檔：52 新版：題目 96 / 講解 97

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

對原防護式測分母為0、商大於3及不大於3的三種情況。

輸入與限制

固定numerator=20；denominator依序0、5、10。

應產生的輸出

三種&&結果為0、1、0；另驗證||跳過右式。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

d=0 result=0
d=5 result=1
d=10 result=0
or=1 right-evaluated=0

63 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

跳至主要內容

不合法時不要執行右側除法。

STEP 02

分母為5時才計算 $20/5 > 3$ ，得到1；分母10時得到0。

STEP 03

用不同觀察變數驗證||左側為真會跳過右側副作用。

原稿解題示例

$d=0 \rightarrow$ 不除法、0； $d=5 \rightarrow 20/5 > 3$ 、1

03 完整程式碼

每份程式均有獨立 main

45_short_circuit.c

完整程式 · 23 行 · 內容與原稿逐字相同

```
/* 45_short_circuit.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 52.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int numerator = 20, denominator = 0;
    int result = denominator != 0 && numerator / denominator > 3;
    printf("d=0 result=%d\n", result);
    denominator = 5;
    result = denominator != 0 && numerator / denominator > 3;
    printf("d=5 result=%d\n", result);
    denominator = 10;
    result = denominator != 0 && numerator / denominator > 3;
    printf("d=10 result=%d\n", result);
    int left_is_true = 1, evaluated = 0;
    int either = left_is_true || (evaluated = 1);
    printf("or=%d right-evaluated=%d\n", either, evaluated);
    return 0;
}
```

單月) 跳至主要內容

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/45_short_circuit.c -o example
```

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

— E43-T01 1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

d=0 result=0
d=5 result=1
d=10 result=0
or=1 right-evaluated=0

63 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

原稿與新增內容的界線

新增固定20與三個分母作可執行測試，並補上原頁提到的||。數值小且商可表示，不宣稱只檢查非零就涵蓋所有整數除法風險。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

分辨逗號運算子與參數分隔符

46_comma_operator.c

1 組原稿測試

原稿索引：46_comma_operator.c 原檔：53 新版：題目 98 / 講解 99

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

執行j=(i++,i+10)，再於另一個printf使用正常參數分隔符。

輸入與限制

i=1；另設a=3,b=4。

應產生的輸出

i=2、j=12，後一行3 4。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

i=2 j=12
3 4

13 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

括號內的逗號先完成左式i++的副作用。

跳至主要內容

再算右式i+10=12，整個運算式以右式作值。

STEP 03

printf的逗號只分隔參數；用無副作用資料示範，不強加先後順序。

原稿解題示例

i=1 → i++完成 → i+10=12 → j=12

03 完整程式碼

每份程式均有獨立 main

46_comma_operator.c

完整程式 · 17 行 · 內容與原稿逐字相同

```
/* 46_comma_operator.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 53.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 1, j;
    j = (i++, i + 10);
    printf("i=%d j=%d\n", i, j);
    int a = 3, b = 4;
    printf("%d %d\n", a, b);
    /* Do not use printf("%d %d", i++, i) to teach ordering. */
    return 0;
}
```

單題編譯指令（使用說明）

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/46_comma_operator.c -o example
```

04 測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列{ 跳至主要內容 結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

– E44-T01 1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

i=2 j=12
3 4

13 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

第一段沿用原例，a=3/b=4為本次新增初始化。反例只留註解。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

比較只計算與真的更新

47_expression_statement.c

1 組原稿測試

原稿索引：47_expression_statement.c 原檔：54 新版：題目 100 / 講解 101

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

從*i*=10開始，分別觀察*i*+1、*i*+=1、*i*=*i*+1是否改變物件。

輸入與限制

固定*i*=10；各次更新與輸出依序執行。

應產生的輸出

只算後10、+=後11、再賦值後12。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

discarded result: *i*=10
compound assignment: *i*=11
simple assignment: *i*=12

73 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

i+1只產生值，未存回，因此*i*仍為10。

跳至主要內容

STEP 02

$i+=1$ 將結果存回，狀態變成 11。

STEP 03

再執行 $i=i+1$ 會再加一次；不要把連續執行當獨立實驗。

原稿解題示例

$i=10 \rightarrow i+1$ 仍 10 $\rightarrow i+=1$ 成 11 $\rightarrow$ 再加成 12

03 完整程式碼

每份程式均有獨立 main

47_expression_statement.c

完整程式 · 19 行 · 內容與原稿逐字相同

```
/* 47_expression_statement.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 54.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int i = 10;
    /* Intentional unused-value warning: the result is discarded. */
    i + 1;
    printf("discarded result: i=%d\n", i);
    i += 1;
    printf("compound assignment: i=%d\n", i);
    i = i + 1;
    printf("simple assignment: i=%d\n", i);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/47_expression_statement.c -o example
```

跳至主要內容

..... 示「刻意保留診斷對照」。程式中的教學反例與警告沒有因格式轉換而刪除；詳細界線見本題第 05 區。

04

測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E45-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字符串核對

返回碼 0 | 完整字符串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

discarded result: i=10
compound assignment: i=11
simple assignment: i=12

73 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原「無作用敘述」刻意保留並產生unused-value警告；此為合法可執行反例，非未定義行為。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

由 UPC 前十一位計算檢查碼

12_upc.c

9 組原稿測試

原稿索引：12_upc.c 原檔：55, 56 新版：題目 102 / 講解 103, 104

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

依原案例的奇偶位置加權規則，計算第十二位應補上的數字。

輸入與限制

前十一位數字；%1d 逐位讀取，允許前導空白，不嚴格驗證整行長度。

應產生的輸出

Check digit: 0..9。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

01380015173

12 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Check digit: 5

15 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

以十一個 %1d 分別保存位置；全部讀成功才計算。

跳至主要內容

STEP 02

從左起奇數位置加總乘 3，偶數位置加總乘 1。

STEP 03

求 $(10 - \text{total} \% 10) \% 10$ ，讓整除10時得到0而非10。

原稿解題示例

01380015173 → total=45 → Check digit: 5

03 完整程式碼

每份程式均有獨立 main

12_upc.c

完整程式 · 24 行 · 內容與原稿逐字相同

```
/* 12_upc.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 55, 56.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int d1, d2, d3, d4, d5, d6, d7, d8, d9, d10, d11;
    int odd, even, total, check;
    if (scanf("%1d%1d%1d%1d%1d%1d%1d%1d%1d%1d",
              &d1, &d2, &d3, &d4, &d5, &d6,
              &d7, &d8, &d9, &d10, &d11) != 11) {
        puts("Input error: enter eleven digits");
        return 1;
    }
    odd = d1 + d3 + d5 + d7 + d9 + d11;
    even = d2 + d4 + d6 + d8 + d10;
    total = 3 * odd + even;
    check = (10 - total % 10) % 10;
    printf("Check digit: %d\n", check);
    return 0;
}
```

單頁 跳至主要內容 月)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/12_upc.c -o example
```

04

測試資料

9 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E46-T01

1. 測試1

exit 0 ✓

原稿測試 · 12 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

01380015173

12 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Check digit: 5

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E46-T02

2. 測試2

exit 0 ✓

原稿測試 · 12 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

00000000000

12 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Check digit: 0

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E46-T03

3. 測試3

exit 0 ✓

原稿測試 · 12 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

跳至主要內容

03600029145

STDOUT · 原稿完整預期輸出

Check digit: 2

12 bytes · 最後有 LF 換行

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E46-T04 4. 測試4

exit 0 ✓

原稿測試 · 14 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

0 13800 15173

14 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Check digit: 5

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E46-T05 5. 測試5

exit 0 ✓

原稿測試 · 12 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

999999999999

12 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Check digit: 3

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E46-T06 6. 測試6

exit 1 ✓

原稿測試 · 12 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

013x0015173

12 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Input error: enter eleven digits

33 bytes · 保留原稿的空白、tab 與換行

跳至主要內容

E46-T07 7. 測試7

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

0138

4 bytes · 最後沒有換行 · 直接 EOF

STDOUT · 原稿完整預期輸出

Input error: enter eleven digits

33 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E46-T08

8. documented-prefix-only

exit 0 ✓

原稿測試 · 13 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

013800151735

13 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Check digit: 5

15 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

—

E46-T09

9. 測試9

exit 1 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

Input error: enter eleven digits

33 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05 來源與注意事項

保留原稿的適用範圍

原稿說明：此線跳至主要內容
保留原稿說明：只消耗前十一個成功數字轉換，不宣稱是嚴格UPC字串驗證。尾端第十二位不檢查，此限制有測試。全零僅是公式邊界資料。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

[跳至主要內容](#)

用零加權和檢查 UPC 公式

48_upc_boundary.c

1 組原稿測試

原稿索引：48_upc_boundary.c 原檔：57 新版：題目 105 / 講解 106

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

分別對total=45、0、58比較原公式與投影片修正版。

輸入與限制

固定三個加權和；不涉及真實商品查詢。

應產生的輸出

一般值都相同；total=0時舊式10、新式0。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

```
total=45 old=5 revised=5
total=0 old=10 revised=0
total=58 old=2 revised=2
```

75 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先讓兩個公式接受同一個total，排除輸入差異。

跳至主要內容

STEP 02

C99中 $-1\%10$ 為 -1 ，因此舊式在 $\text{total}=0$ 得到10。

STEP 03

修正式再取 $\%10$ ，讓檢查碼維持一位數範圍。

原稿解題示例

$\text{total}=0 \rightarrow \text{old}=10$ 、 $\text{revised}=0$

03 完整程式碼

每份程式均有獨立 main

48_upc_boundary.c

完整程式 · 17 行 · 內容與原稿逐字相同

```
/* 48_upc_boundary.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 57.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int total = 45;
    printf("total=%d old=%d revised=%d\n", total, 9-((total-1)%10), (10-total%10)%10);
    total = 0;
    printf("total=%d old=%d revised=%d\n", total, 9-((total-1)%10), (10-total%10)%10);
    total = 58;
    printf("total=%d old=%d revised=%d\n", total, 9-((total-1)%10), (10-total%10)%10);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/48_upc_boundary.c -o example
```

0 跳至主要內容 ↓

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	total=45 old=5 revised=5 total=0 old=10 revised=0 total=58 old=2 revised=2
0 bytes · 空輸入檔 · 直接 EOF	75 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

原第57頁兩公式與三筆測試完整保留；舊式是結果不合規格的合法計算，可以安全比較。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

修正平均值程式的三個問題

49_debug_io_fixed.c

3 組原稿測試

原稿索引：49_debug_io_fixed.c 原檔：58 新版：題目 107 / 講解 108

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

修正整數除法、printf格式與scanf目標型別配對，再執行驗證。

輸入與限制

先算80與81平均；再讀一個double，例如82.5。

應產生的輸出

calculated=80.5；read=82.5。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

82.5

5 bytes · 最後有 LF 換行

STDOUT · 範例輸出

calculated=80.5
read=82.5

26 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

計算階段把/2改成/2.0，讓除法保留小數。

跳至主要內容

輸出階段用%.1f匹配double，不用%d。

STEP 03

讀入階段用%lf與&average，回傳值1才印新值。

原稿解題示例

先算80.5；再輸入82.5 → 新值82.5

03 完整程式碼

每份程式均有獨立 main

49_debug_io_fixed.c

完整程式 · 24 行 · 內容與原稿逐字相同

```
/* 49_debug_io_fixed.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 58.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#include <math.h>

int main(void)
{
    double average = (80 + 81) / 2.0;
    printf("calculated=%.1f\n", average);
    if (scanf("%lf", &average) != 1) {
        puts("Input error");
        return 1;
    }
    if (!isfinite(average) || average < -10000.0 || average > 10000.0) {
        puts("Range error");
        return 1;
    }
    printf("read=%.1f\n", average);
    /* Original wrong-type calls remain in the teaching slide, not executed here. */
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/49_debug_io_fixed.c -o example
```

[跳至主要內容](#)

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

E48-T01 1. 測試1 原稿測試 · 5 input bytes · 完整字串核對	exit 0 ✓
返回碼 0 完整字串核對 STDIN · 完整輸入 82.5 5 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 calculated=80.5 read=82.5 26 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E48-T02 2. 測試2 原稿測試 · 4 input bytes · 完整字串核對	exit 1 ✓
返回碼 1 完整字串核對 STDIN · 完整輸入 abc 4 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 calculated=80.5 Input error 28 bytes · 保留原稿的空白、tab 與換行 原稿測試標記：PASS	
E48-T03 3. 測試3 原稿測試 · 4 input bytes · 完整字串核對	exit 1 ✓
返回碼 1 完整字串核對 STDIN · 完整輸入 nan 跳至主要內容 4 bytes · 最後有 LF 換行 STDOUT · 原稿完整預期輸出 calculated=80.5 Range error 28 bytes · 保留原稿的空白、tab 與換行	

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

本次提供原除錯題的明確修正版，不執行原不匹配格式。讀入值±10000與有限值防護為新增教學範圍。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

計算以半小時為單位的停車費

13_parking.c

12 組原稿測試

原稿索引：13_parking.c 原檔：59, 60, 61, 62 新版：題目 109 / 講解 110, 111, 112, 113

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

每開始30分鐘收20元，0分鐘不收費；將規格轉為可測試公式。

輸入與限制

一個整數分鐘數，0..1440；費率僅為教材假設。

應產生的輸出

Minutes、Blocks、Fee，各一行。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 範例輸出

Minutes: 0

Blocks: 0

Fee: 0

28 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

先分開處理讀取失敗與超出範圍，錯誤時不進入計算。

跳至主要內容

STEP 02

以 $(\text{minutes} + 29) / 30$ 計算開始過的半小時單位。

STEP 03

單位數乘20得費用；先核對0、30、31三個邊界。

原稿解題示例

31 → Blocks: 2 → Fee: 40

03 完整程式碼

每份程式均有獨立 main

13_parking.c

完整程式 · 25 行 · 內容與原稿逐字相同

```
/* 13_parking.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 59, 60, 61, 62.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>
#define BLOCK_MINUTES 30
#define PRICE_PER_BLOCK 20

int main(void)
{
    int minutes, blocks, fee;
    if (scanf("%d", &minutes) != 1) {
        puts("Input error");
        return 1;
    }
    if (minutes < 0 || minutes > 1440) {
        puts("Range error");
        return 1;
    }
    blocks = (minutes + BLOCK_MINUTES - 1) / BLOCK_MINUTES;
    fee = blocks * PRICE_PER_BLOCK;
    printf("Minutes: %d\nBlocks: %d\nFee: %d\n", minutes, blocks, fee);
    return 0;
}
```

[跳至主要內容](#)

[單題編譯指令 \(使用說明\)](#)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/13_parking.c -o example
```

04

測試資料

12 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

–

E49-T01

1. 測試1

exit 0 ✓

原稿測試 · 2 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

0

2 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Minutes: 0
Blocks: 0
Fee: 0

28 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E49-T02

2. 測試2

exit 0 ✓

原稿測試 · 2 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

1

2 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Minutes: 1
Blocks: 1
Fee: 20

29 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E49-T03

3. 測試3

exit 0 ✓

原稿測試 · 3 input bytes · 完整字串核對

跳至主要內容

STDIN · 完整輸入

STDOUT · 原稿完整預期輸出

29

3 bytes · 最後有 LF 換行

Minutes: 29
Blocks: 1
Fee: 20

30 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E49-T04 4. 測試4

exit 0 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

30

3 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Minutes: 30
Blocks: 1
Fee: 20

30 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E49-T05 5. 測試5

exit 0 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

31

3 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Minutes: 31
Blocks: 2
Fee: 40

30 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E49-T06 6. 測試6

exit 0 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

跳至主要內容

90

STDOUT · 原稿完整預期輸出

Minutes: 90
Blocks: 3

3 bytes · 最後有 LF 換行

Fee: 60

30 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E49-T07 7. 測試7

exit 0 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

1440

5 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Minutes: 1440

Blocks: 48

Fee: 960

34 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E49-T08 8. 測試8

exit 1 ✓

原稿測試 · 3 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

-1

3 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E49-T09 9. 測試9

exit 1 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入

1441

跳至主要內容

3 bytes · 最後有 LF 換行

STDOUT · 原稿完整預期輸出

Range error

12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E49-T10

10. 測試10

exit 1 ✓

原稿測試 · 4 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
abc	Input error
4 bytes · 最後有 LF 換行	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E49-T11

11. 測試11

exit 1 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
(空輸入檔 · 直接 EOF)	Input error
0 bytes · 空輸入檔 · 直接 EOF	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

–

E49-T12

12. documented-prefix-only

exit 0 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
31.5	Minutes: 31
5 bytes · 最後有 LF 換行	Blocks: 2
	Fee: 40
	30 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

跳至主要內容

原稿與新增內容的界線

保留原程式的scanf整數前綴解析。輸入31.5時%d讀到31，未驗證尾端，因此本版不宣稱會拒絕所有小數字串。這是原教學程式的範圍，不新增免費時段或每日封頂。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

[跳至主要內容](#)

驗證可計算的總檢核答案

50_quiz_integrated.c

1 組原稿測試

原稿索引：50_quiz_integrated.c 原檔：63 新版：題目 114 / 講解 115

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

用固定測試核對停車費、double讀入及多餘參數；未定義反例只分類。

輸入與限制

31分鐘、固定字串3.5；i*i++不執行。

應產生的輸出

blocks=2、fee=40、count=1、x=3.5、多餘參數只印10。

輸入 / 輸出範例

沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

```
blocks=2 fee=40
count=1 x=3.5
i*i++: undefined behavior; not executed.
extra argument example=10
```

97 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

跳至主要內容

可推導答案，避免對未定義式硬算。

STEP 02

可計算部分分別用固定資料驗證，避免互相干擾。

STEP 03

對多餘printf參數觀察警告與有效輸出，不與少參數混為一談。

原稿解題示例

31分鐘 → 2單位 / 40 ; i*i++ → 不給固定輸出

03 完整程式碼

每份程式均有獨立 main

50_quiz_integrated.c

完整程式 · 19 行 · 內容與原稿逐字相同

```
/* 50_quiz_integrated.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 63.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int blocks = (31 + 29) / 30;
    printf("blocks=%d fee=%d\n", blocks, blocks * 20);
    double x = 0.0;
    int count = sscanf("3.5", "%lf", &x);
    printf("count=%d x=%.1f\n", count, x);
    puts("i*i++: undefined behavior; not executed.");
    /* Intentional format-extra-args warning. */
    printf("extra argument example=%d\n", 10, 20);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/50_quiz_integrated.c -o example
```

跳至主要內容

..... 示「刻意保留診斷對照」。程式中的教學反例與警告沒有因格式轉換而刪除；詳細界線見本題第 05 區。

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

—

E50-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

blocks=2 fee=40
count=1 x=3.5
i*i++: undefined behavior; not executed.
extra argument example=10

97 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

沿用原四題答案；sscanf與固定3.5是本次測試支援。多餘參數範例有預期警告。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

file:///C:/Users/Alvin/Documents/Codex/2026-10-04/new-chat-2/work/rendered/CH1-4_%E5%AE%8C%E6%95%B4%E7%A8%8B%E5%BC%8F...

177/194

觀察 scanf 星號的抑制賦值

51_scan_suppression.c

4 組原稿測試

原稿索引：51_scan_suppression.c 原檔：64 新版：題目 116 / 講解 117

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

讀兩個整數但只保存第二個，對照printf星號的完全不同用途。

輸入與限制

輸入10 20；格式%*d%d只提供&x。

應產生的輸出

count=1、x=20；動態輸出欄寬仍由另一個參數提供。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

10 20

6 bytes · 最後有 LF 換行

STDOUT · 範例輸出

count=1 x=20
| 20|

21 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

%*d先讀第一個整數但不賦值，因此不提供第一個寫入位置。

跳至主要內容

第二個%d才寫入x，scanf成功賦值數是1而非2。

STEP 03

再以printf的%*d示範動態欄寬，不把兩個星號混用。

原稿解題示例

10 20 → count=1 、 x=20 → | 20|

03 完整程式碼

每份程式均有獨立 main

51_scan_suppression.c

完整程式 · 19 行 · 內容與原稿逐字相同

```
/* 51_scan_suppression.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 64.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    int x = -1;
    int count = scanf("%d%d", &x);
    if (count != 1) {
        puts("Input error");
        return 1;
    }
    printf("count=%d x=%d\n", count, x);
    printf("|%d|\n", 5, x);
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/51_scan_suppression.c -o example
```

04 測試資料

4 組完整原稿資料

輸入，跳至主要內容 用原稿；來源觀察值另行標示。

列印版列出測試系列與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
<pre>10 20</pre>	<pre>count=1 x=20 20 </pre>
6 bytes · 最後有 LF 換行	21 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E51-T02 2. 測試2

exit 0 ✓

原稿測試 · 6 input bytes · 完整字串核對

返回碼 0 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
<pre>10 -7</pre>	<pre>count=1 x=-7 -7 </pre>
6 bytes · 最後有 LF 換行	21 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E51-T03 3. 測試3

exit 1 ✓

原稿測試 · 5 input bytes · 完整字串核對

返回碼 1 | 完整字串核對

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出
<pre>10 x</pre>	<pre>Input error</pre>
5 bytes · 最後有 LF 換行	12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

E51-T04 4. 測試4

exit 1 ✓

原稿測試 · 0 input bytes · 完整字串核對

跳至主要內容

↵

STDIN · 完整輸入	STDOUT · 原稿完整預期輸出

(空輸入檔，直接 EOF)
0 bytes · 空輸入檔，直接 EOF

Input error
12 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

將原附錄備忘錄的%*d%d題實作；printf的動態欄寬是用來作跨函式對照的新增行。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

觀察數值跳脫序列的邊界

52_numeric_escapes.c

1 組原稿測試

原稿索引：52_numeric_escapes.c 原檔：65 新版：題目 118 / 講解 119

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

在ASCII相容執行字元集上，輸出八進位A與明確截斷的十六進位AB。

輸入與限制

無輸入；檢查A/B是否為ASCII對應值。

應產生的輸出

A、AB，另示範水平跳格。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 範例輸出

A
AB
tab:A B

13 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

STEP 01

\101採最多三個八進位數字；此例值為65。

跳至主要內容

STEP 02

把\x41與後面的B分成相鄰字串，避免B被十六進位序列吸收。

STEP 03

輸出前先檢查字元集前提；控制字元的顯示仍由終端決定。

原稿解題示例

"\101" → A ; "\x41" "B" → AB

03 完整程式碼

每份程式均有獨立 main

52_numeric_escapes.c

完整程式 · 19 行 · 內容與原稿逐字相同

```
/* 52_numeric_escapes.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 65.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    if ('A' != 65 || 'B' != 66) {
        puts("ASCII-compatible execution character set required for this demo.");
        return 0;
    }
    printf("%s\n", "\101");
    printf("%s\n", "\x41" "B");
    printf("tab:A\tB\n");
    /* '\a', '\b', and '\r' are listed in the guide, not used for visual assertions. */
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/52_numeric_escapes.c -o example
```

[跳至主要內容](#)

04

測試資料

1 組完整原稿資料

輸入、輸出與返回碼沿用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

– E52-T01

1. 測試1

exit 0 ✓

原稿測試 · 0 input bytes · 完整字符串核對

返回碼 0 | 完整字符串核對

STDIN · 完整輸入

(空輸入檔 · 直接 EOF)

0 bytes · 空輸入檔 · 直接 EOF

STDOUT · 原稿完整預期輸出

A
AB
tab:A B

13 bytes · 保留原稿的空白、tab 與換行

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

數值序列沿用原附錄；新增執行字元集檢查與tab示範。ASCII以外環境會印前提訊息，而不是保證A/AB輸出。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

跳至主要內容

用不同進位顯示同一個值

14_format_extras.c

1 組原稿測試

原稿索引：14_format_extras.c 原檔：66 新版：題目 120 / 講解 121

格式轉換：題目、三步解法、程式與測試資料均沿用上傳 HTML；原有順序與檔名不變。

01 完整題目

先確認規格，再寫程式

觀察十進位、十六進位、八進位與十六進位浮點輸出。

輸入與限制

固定 unsigned int n=26u、double 1.5。

應產生的輸出

26 0x1a 032；%a 的等值表示；字元、字串與百分號。

輸入 / 輸出範例 沿用本題原稿第一組測試

STDIN · 範例輸入

(空輸入檔，直接 EOF)

0 bytes · 空輸入檔，直接 EOF

STDOUT · 原稿平台觀察值

26 0x1a 032

0x1.800p+0

A Hello %

33 bytes · 保留原稿的空白、tab 與換行

空輸入檔代表直接 EOF，不需輸入「EOF」文字；下載保留實際空白、tab 及有無末尾換行。

此例含平台觀察值，請依原題前提判讀，不將本機型別範圍或浮點字面表示當成跨平台固定答案。

02 解題說明

沿用原稿的核心思考

核心解法的思考邏輯

依原文逐步閱讀；按鈕只標記解題文字，不會執行 C 程式，也不額外指定運算式求值順序。

跳至主要內容

STEP 01

先確認型別：%u、%x、%o 都使用 unsigned int。

STEP 02

同一數值用不同格式顯示，不改變 n 本身。

STEP 03

辨識 %a 的 p 是以2為底的指數；%% 不消耗數值參數。

原稿解題示例

26u → 26、0x1a、032

03 完整程式碼

每份程式均有獨立 main

14_format_extras.c

完整程式 · 15 行 · 內容與原稿逐字相同

```
/* 14_format_extras.c
 * Source: C_Programming_Ch1-4.pptx, actual slide(s) 66.
 * This is a complete teaching edition; see the accompanying Chinese guide.
 * Build each .c file separately. No project-specific headers are required.
 */
#include <stdio.h>

int main(void)
{
    unsigned int n = 26u;
    printf("%u %x %o\n", n, n, n);
    printf("%.3a\n", 1.5);
    printf("%c %s %%\n", 'A', "Hello");
    return 0;
}
```

單題編譯指令 (使用說明)

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/14_format_extras.c -o example
```

04 測試資料

1 組完整原稿資料

跳至主要內容

輸入、輸出、觀察值均採用原稿；來源觀察值另行標示。

列印版列出測試索引與結束碼；完整輸入、輸出及原稿核對說明請使用 HTML 或下載的測試檔。

返回碼 0 | 依模式或數值核對；以下列出本次GCC環境觀察值

STDIN · 完整輸入	STDOUT · 原稿平台觀察值
(空輸入檔 · 直接 EOF)	26 0x1a 032
	0x1.800p+0
0 bytes · 空輸入檔 · 直接 EOF	A Hello %
	33 bytes · 保留原稿的空白、tab 與換行

上方是原 HTML 記錄的 GCC 觀察值，本次環境輸出與其一致；不是所有平台都必須逐字相同的答案。原稿未附具體模式 / 數值判準，本次未補寫該判準。

原稿測試標記：**PASS**

05

來源與注意事項

保留原稿的適用範圍

原稿與新增內容的界線

保留原程式；%a測試接受實作所選等值十六進位表示，並另檢驗其數值為1.5，不以本機字面輸出限制所有C實作。

本區「本次 / 新增」等措辭來自原 HTML 的逐題說明，並非本次版型轉換新增題目或修改 C 程式的宣稱。

HOW TO USE / CONVERSION VALIDATION

使用與驗證說明

CH1–4 原有內容，套用 CH5–7 的閱讀與下載介面。

這份 HTML 保留了什麼？

53 個範例、53 份完整 C 程式、133 組測試資料，原有題目、輸入與輸出規格、三步解法、解題示例、逐題來源界線、原稿頁次及閱讀順序均保留。介面群組是檢索用的主題分類，未另行宣稱原始章節分界。

文! 跳至主要內容! 為圖片。程式、輸入檔、輸出檔與 ZIP 都內嵌於同一 HTML；即使離線，也不必連到原稿相對路徑中的code/ 或 README.md。

「隱藏解答」只是顯示模式；題目範例仍屬題目的一部分，完整解答及下載資料也仍存放在檔案內。「下一步」只標記原稿三步解法，不在瀏覽器中執行 C，也不額外宣稱語言的求值次序。按 Esc 可關閉手機目錄，按 / 可聚焦搜尋。

內容來源與本次轉換界線

內容來自上傳的 `CH1-4_完整程式碼與解題說明.html`；視覺與互動介面沿用 `C_Programming_Ch5-7_Examples.html`。本次未補入新題目、新解法或原稿缺少的變數表，也未修正 C 演算法與輸入規則。

每題第 05 區保留原 HTML 的「原稿與新增內容的界線」。其中「本次 / 新增」沿用原稿的說明，不是這次版型轉換新增程式內容。原稿標示的 122 頁投影片及各頁索引照錄，本次未重新讀取 CH1-4 PPTX 核對頁次。

原 HTML 提及的 README 及原始測試器未包含在此次上傳檔案中。本頁、資源包 README 與 `tests/run_tests.py` 是本次另製的操作 / 轉換驗證支援，不冒充原有文件。

單題編譯與執行

下載頂部「程式與測試 ZIP」並完整解壓縮。每個檔案各有 `main`，請分開編譯，不要將53支程式連結成同一支執行檔。

Linux / macOS shell

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/02_dweight.c -o example
./example < tests/data/E07/E07-T01.in.txt
echo $?
```

Windows 命令提示字元 (cmd.exe) 使用範例

```
gcc -std=c11 -Wall -Wextra -Wpedantic code/02_dweight.c -o example.exe
example.exe < tests/data/E07/E07-T01.in.txt
echo %ERRORLEVEL%
```

Windows 指令供操作參考，本次未在 Windows 實測。上列重導向寫法屬 cmd.exe，不是 PowerShell；需先具備編譯器，HTML 本身不會安裝或執行編譯器。

本次實際重跑的範圍

以原程式逐份編譯，再將原稿的133組輸入交給對應執行檔。本次 Linux 環境中，三輪結果如下。

跳至主要內容

實測工具	程式編譯	原稿資料核對
GCC 14.2	53 / 53	133 / 133
Clang 17.0	53 / 53	133 / 133
GCC 14.2 + UBSan	53 / 53	133 / 133

核對完整標準輸出、程序結束碼及空的 `stderr`；只將 CRLF 正規化為 LF，不刪除其他空白。刻意產生的編譯警告保留於報告，未使用 `-Werror`，也沒有為消除警告而更改程式。完整編譯輸出與每組實測結果包含在資源 ZIP 的 `validation/`。

3 組平台觀察值不是跨平台固定答案。

`18_basic_types.c`、`43_behavior_classes.c`、`14_format_extras.c` 的原稿標示「依模式或數值核對」，但未附具體判準。本次只核對現有環境與原稿保存的完整觀察字串相同，沒有補寫未知判準。其他平台的型別範圍或等值十六進位浮點表示可能不同，應依原題前提判讀。

重跑測試

```
python tests/run_tests.py --compiler gcc --report local_gcc.json
python tests/run_tests.py --compiler clang --report local_clang.json
python tests/run_tests.py --compiler gcc --sanitize --report local_ubsan.json
```

本次新增測試工具使用 Python 標準函式庫，需 Python 3.9 以上及相應編譯器。它只比對所附原稿字串，不宣稱重建原先未附的模式 / 數值測試器。不同平台的觀察資料不一致時須另行判讀。

空檔、空白行與資料末尾

0 bytes 是直接 EOF；只有 的輸入是一個空白行。測試區可切換「顯示跳脫字元」，辨識 tab、空白及末尾換行；下載永遠使用原始資料，不是 JSON 跳脫後的顯示文字。

`31_scan_trailing_space.c` 是原稿刻意安排的互動等待實驗；人工操作前請閱讀其來源提醒。自動測試在送完輸入後關閉串流，與終端只按 Enter 仍持續開啟串流的情境不同。

原稿共通限制仍適用

數值須可由對應型別表示；scanf 前綴解析不等於嚴格整行驗證。原稿的安全改寫、ASCII及平台前提、教學費率與刻意診斷均不變。測試通過不是對任意輸入的形式證明，也不是 Windows、MSVC 或 Office 實測。

列印目前頁只印正在閱讀的範例；列印全冊包含所有53題。為控制篇幅，列印版的測試區列出索引及結束碼，完整輸入 / 輸出請使用 跳至主要內容 。

[查看來源索引](#) →

來源與案例索引

原稿順序、檔名、頁次與新增界線完整保留。

這次使用的兩份 HTML

內容來源

CH1-4_完整程式碼與解題說明.html

53題、53程式、133組原稿資料。

版型與操作來源

C_Programming_Ch5-7_Examples.html

側邊目錄、主題卡片、分區、配色與互動方式。

保留比對：原題文字、輸入 / 輸出規格、三步解法、解題示例、來源段落、程式碼及測試資料逐項核對。範例E01-E53只是沿用原稿閱讀序列的顯示標籤；不更名或重排任何C檔案。

原 HTML 的使用說明（保留原文）

下方「本次實測」等用語屬原稿文字。本次轉換的重跑方法及限制另列於「使用與驗證說明」。原稿中的 README 超連結未附文件，這裡以純文字保留。

使用前先確認

每個檔案各有 `main`，請分開編譯；完整限制與指令見 README。GCC、Clang、GCC + UBSan 均通過 53 檔編譯及 133 組測試。下列結果以本次實測為依據；特定型別大小、字元編碼與浮點表示須依題目前提判讀。

原投影片的非法 / 未定義反例只作分析；程式包提供明確標示的安全改寫，並不宣稱與未定義原式等價。scanf 範例要求數值可由型別表示，不保證嚴格整行解析。

原 HTML 的來源宣告（照錄）：來源：本次上傳的 C_Programming_Ch1-4.pptx。新增程式與講解均逐題標記。

跳至主要內容

完整範例對照

序號	題目 / 原程式	原稿頁次記錄	資料
E01	分辨三類錯誤並執行修正版 16_diagnostics_safe.c	16_diagnostics_safe.c 原檔：4 新版：題目 6 / 講解 7	1 組
E02	建立並執行第一支 C 程式 01_hello.c	01_hello.c 原檔：5, 6 新版：題目 8 / 講解 9, 10	1 組
E03	以名稱與註解表達計算意圖 17_naming.c	17_naming.c 原檔：7 新版：題目 11 / 講解 12	1 組
E04	觀察型別與實作範圍 18_basic_types.c	18_basic_types.c 原檔：8 新版：題目 13 / 講解 14	1 組
E05	安全觀察初始化與賦值 19_initialization.c	19_initialization.c 原檔：9 新版：題目 15 / 講解 16	1 組
E06	用中間值追蹤體積計算 20_volume_flow.c	20_volume_flow.c 原檔：10 新版：題目 17 / 講解 18	1 組
E07	計算包裹的材積重量 02_dweight.c	02_dweight.c 原檔：11, 13 新版：題目 19 / 講解 20, 23	7 組
E08	驗證整數向上取整的邊界 21_ceiling_division.c	21_ceiling_division.c 原檔：12 新版：題目 21 / 講解 22	8 組
E09	比較巨集有無括號的結果 22_macro_parentheses.c	22_macro_parentheses.c 原檔：14 新版：題目 24 / 講解 25	1 組
E10	華氏溫度轉攝氏溫度 03_celsius.c	03_celsius.c 原檔：15, 16 新版：題目 26 / 講解 27, 28	9 組
E11	驗證基本元素的四題檢核 23_quiz_basics.c	23_quiz_basics.c 原檔：17 新版：題目 29 / 講解 30	1 組
E12	讓格式與數值一一配對 24_printf_contract.c	24_printf_contract.c 原檔：18 新版：題目 31 / 講解 32	1 組
E13	依資料型別選擇 printf 規格 25_printf_types.c	25_printf_types.c 原檔：19 新版：題目 33 / 講解 34	1 組
E14	比較欄寬、精度與浮點格式 04_printf_formats.c	04_printf_formats.c 原檔：20, 21 新版：題目 35 / 講解 36, 37	1 組
E	跳至主要內容 感測器數值報表 05_sensor_report.c	05_sensor_report.c 原檔：22 新版：題目 38 / 講解 39	1 組

序號	題目 / 原程式	原稿頁次記錄	資料
E16	控制字串截取與動態欄寬 26_dynamic_format.c	26_dynamic_format.c 原檔：23 新版：題目 40 / 講解 41	1 組
E17	輸出引號、反斜線與換行 27_escape_sequences.c	27_escape_sequences.c 原檔：24 新版：題目 42 / 講解 43	1 組
E18	區分多給參數與危險格式錯誤 28_printf_extra_args.c	28_printf_extra_args.c 原檔：25 新版：題目 44 / 講解 45	1 組
E19	正確讀取 float 與 double 29_scan_value_address.c	29_scan_value_address.c 原檔：26 新版：題目 46 / 講解 47	4 組
E20	逐項解析沒有空白的數字串 06_scan_stream.c	06_scan_stream.c 原檔：27, 28 新版：題目 48 / 講解 49, 50	5 組
E21	觀察 scanf 部分成功後的狀態 07_scan_failure.c	07_scan_failure.c 原檔：29 新版：題目 51 / 講解 52	4 組
E22	重現五組分隔符匹配實驗 30_scan_delimiters.c	30_scan_delimiters.c 原檔：30 新版：題目 53 / 講解 54	1 組
E23	比較 %c 與前導空白的讀取 08_scan_char.c	08_scan_char.c 原檔：31 新版：題目 55 / 講解 56	4 組
E24	親自觀察尾端空白指令的等待 31_scan_trailing_space.c	31_scan_trailing_space.c 原檔：32 新版：題目 57 / 講解 58	3 組
E25	將兩個分數相加且先約分 09_fractions.c	09_fractions.c 原檔：33, 34 新版：題目 59 / 講解 60, 61	8 組
E26	用小實驗驗證四題格式化檢核 32_quiz_io.c	32_quiz_io.c 原檔：35 新版：題目 62 / 講解 63	1 組
E27	拆解一元與二元運算 33_unary_binary.c	33_unary_binary.c 原檔：36 新版：題目 64 / 講解 65	1 組
E28	比較兩筆量測值的平均 34_average_division.c	34_average_division.c 原檔：37 新版：題目 66 / 講解 67	1 組
E29	驗證正負整數的商與餘數 35_quotient_remainder.c	35_quotient_remainder.c 原檔：38 新版：題目 68 / 講解 69	1 組
E30	把總秒數拆成時、分、秒 10_duration.c	10_duration.c 原檔：39 新版：題目 70 / 講解 71	9 組
E	跳至主要內容 括號改變前後的運算樹 36_precedence.c	36_precedence.c 原檔：40 新版：題目 72 / 講解 73	1 組

序號	題目 / 原程式	原稿頁次記錄	資料
E32	用括號驗證結合性 37_associativity.c	37_associativity.c 原檔：41 新版：題目 74 / 講解 75	1 組
E33	追蹤連鎖賦值的型別轉換 38_assignment_value.c	38_assignment_value.c 原檔：42 新版：題目 76 / 講解 77	1 組
E34	辨識可修改的賦值目標 39_lvalue_safe.c	39_lvalue_safe.c 原檔：43 新版：題目 78 / 講解 79	1 組
E35	追蹤庫存與複合賦值 40_compound_assignment.c	40_compound_assignment.c 原檔：44 新版：題目 80 / 講解 81	1 組
E36	分開觀察遞增的值與狀態 41_pre_post_increment.c	41_pre_post_increment.c 原檔：45 新版：題目 82 / 講解 83	1 組
E37	追蹤三組獨立的遞增運算式 11_expression_trace.c	11_expression_trace.c 原檔：46 新版：題目 84 / 講解 85	1 組
E38	逐層解出複合運算式 53_complex_expression.c	53_complex_expression.c 原檔：47 新版：題目 86 / 講解 87	1 組
E39	分開驗證分組與人為指定順序 42_grouping_order.c	42_grouping_order.c 原檔：48 新版：題目 88 / 講解 89	1 組
E40	將四種語言行為分開說明 43_behavior_classes.c	43_behavior_classes.c 原檔：49 新版：題目 90 / 講解 91	1 組
E41	把衝突副作用改成明確步驟 15_safe_rewrite.c	15_safe_rewrite.c 原檔：50 新版：題目 92 / 講解 93	1 組
E42	正確檢查分數是否在範圍內 44_relational_logical.c	44_relational_logical.c 原檔：51 新版：題目 94 / 講解 95	6 組
E43	用短路規則保護除法 45_short_circuit.c	45_short_circuit.c 原檔：52 新版：題目 96 / 講解 97	1 組
E44	分辨逗號運算子與參數分隔符 46_comma_operator.c	46_comma_operator.c 原檔：53 新版：題目 98 / 講解 99	1 組
E45	比較只計算與真的更新 47_expression_statement.c	47_expression_statement.c 原檔：54 新版：題目 100 / 講解 101	1 組
E46	由 UPC 前十一位計算檢查碼 12_upc.c	12_upc.c 原檔：55, 56 新版：題目 102 / 講解 103, 104	9 組
E	跳至主要內容 加權和檢查 UPC 公式 48_upc_boundary.c	48_upc_boundary.c 原檔：57 新版：題目 105 / 講解 106	1 組

序號	題目 / 原程式	原稿頁次記錄	資料
E48	修正平均值程式的三個問題 49_debug_io_fixed.c	49_debug_io_fixed.c 原檔：58 新版：題目 107 / 講解 108	3 組
E49	計算以半小時為單位的停車費 13_parking.c	13_parking.c 原檔：59, 60, 61, 62 新版：題目 109 / 講解 110, 111, 112, 113	12 組
E50	驗證可計算的總檢核答案 50_quiz_integrated.c	50_quiz_integrated.c 原檔：63 新版：題目 114 / 講解 115	1 組
E51	觀察 scanf 星號的抑制賦值 51_scan_suppression.c	51_scan_suppression.c 原檔：64 新版：題目 116 / 講解 117	4 組
E52	觀察數值跳脫序列的邊界 52_numeric_escapes.c	52_numeric_escapes.c 原檔：65 新版：題目 118 / 講解 119	1 組
E53	用不同進位顯示同一個值 14_format_extras.c	14_format_extras.c 原檔：66 新版：題目 120 / 講解 121	1 組

原稿頁次照錄，未將未附於本次任務的 CH1–4 投影片當成已重新核對的來源。